

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 6) FULLSCREEN AND RESIZING

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

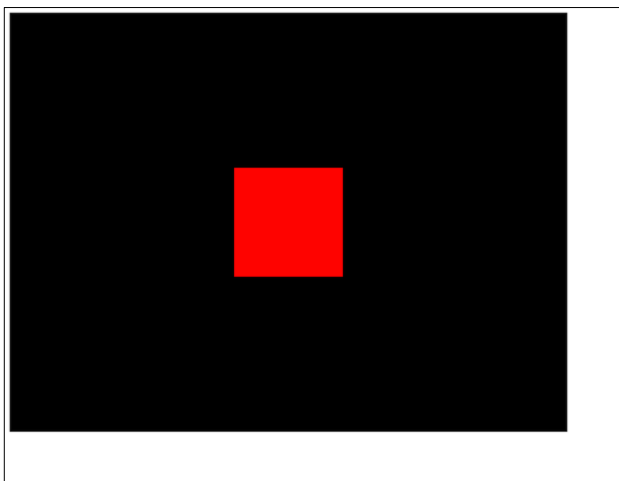
### Introduction

Our canvas currently has a fixed resolution of **800x600**. You don't necessarily need your WebGL to fit the whole screen, but it might be better if you want an immersive experience.

First, we would like to have the canvas take all available space. Then, we must ensure it still fits if the user resizes their window. Finally, we must allow the user to experiment with the full-screen experience.

### Setup

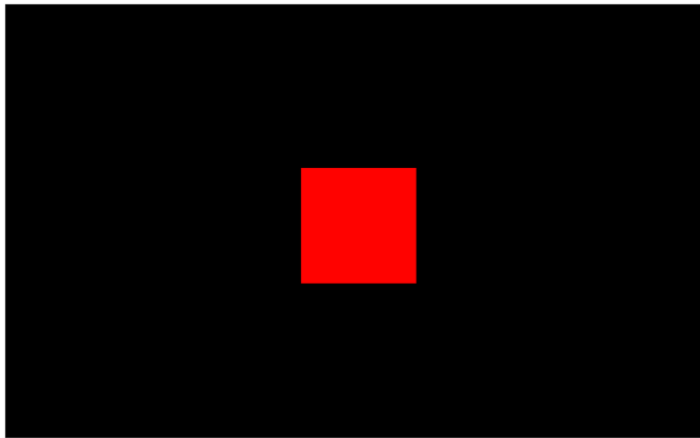
The starter contains what we finished in the previous lesson. We have our cube in the center, and we can drag and drop to move the camera.



### Fit in the viewport

Instead of using fixed numbers in the sizes variable, use the `window.innerWidth` and `window` to make the canvas fit perfectly in the viewport.**innerHeight**:

```
// ...  
  
// Sizes  
const sizes = {  
  width: window.innerWidth,  
  height: window.innerHeight  
}  
  
// ...
```



You can see that the canvas now has the width and height of the viewport. Unfortunately, there is a white margin and a scroll bar (try to scroll if you don't see any scrollbar).

The problem is that browsers all have default stylings like more significant titles, underlined links, space between paragraphs, and paddings on the page. There are many ways of fixing that, and it might depend on the rest of your website. If you have other content, try not to break any of those while doing this.

We will keep things simple and fix the position of the canvas using CSS.

Our HTML is already loading the **src/style.css** file:

```
<link rel="stylesheet" href="./style.css">
```

You can write standard CSS just like you're used to, and the page will automatically reload.

A good thing to do first would be to remove any type of **margin** or **padding** on all elements by using a wildcard \*:

```
*  
{  
  margin: 0;  
  padding: 0;  
}
```

Then, we can fix the canvas on the top left using its **webgl** class to select it:

```
.webgl  
{  
  position: fixed;  
  top: 0;  
  left: 0;  
}
```

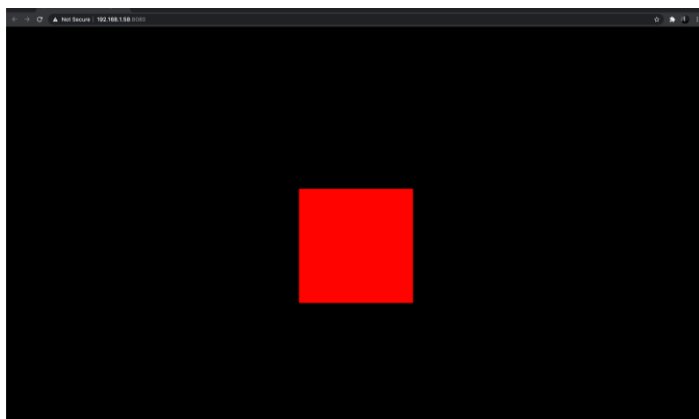
You don't need to specify **width** or **height** on the canvas because Three.js already takes care of that when you call the **renderer.setSize(...)** method.

This is a good opportunity to fix a small problem on our canvas. You may have noticed a blue outline on it when dragging and dropping. This mostly happens on the latest versions of Chrome. To fix that, we can simply add an **outline: none;** on the **.webgl**:

```
.webgl
{
  position: fixed;
  top: 0;
  left: 0;
  outline: none;
}
```

If you want to remove any type of scrolling even on touch screens, you can add an **overflow: hidden** on both **html** and **body**:

```
html,
body
{
  overflow: hidden;
}
```



You can now enjoy your WebGL in all its glory. Unfortunately, if you resize the window, the canvas won't follow.

We need to deal with the resize.

## Handle resize

To resize the canvas, we first need to know when the window is being resized. To do so, you can listen to the **resize** event on the window.

Add the **resize** listener right after the **sizes** variable:

```
window.addEventListener('resize', () =>
{
  console.log('window has been resized')
})
```

Now that we trigger a function when the window is being resized, we need to update a few things in our code.

First, we must update the **sizes** variable:

```
window.addEventListener('resize', () =>
{
  // Update sizes
  sizes.width = window.innerWidth
  sizes.height = window.innerHeight
})
```

Secondly, we must update the **camera** aspect ratio by changing its **aspect** property:

```
window.addEventListener('resize', () =>
{
  // ...

  // Update camera
  camera.aspect = sizes.width / sizes.height
})
```

When you change camera properties like **aspect** you also need to update the projection matrix using **camera.updateProjectionMatrix()**. We will talk about matrices later:

```
window.addEventListener('resize', () =>
{
  // ...

  camera.updateProjectionMatrix()
})
```

Finally, we must update the **renderer**. Updating the renderer will automatically update the canvas width and height:

```
window.addEventListener('resize', () =>
{
  // ...

  // Update renderer
  renderer.setSize(sizes.width, sizes.height)
})
```

All together:

```
window.addEventListener('resize', () =>
{
  // Update sizes
  sizes.width = window.innerWidth
  sizes.height = window.innerHeight

  // Update camera
  camera.aspect = sizes.width / sizes.height
```

```
camera.updateProjectionMatrix()

// Update renderer
renderer.setSize(sizes.width, sizes.height)
})
```

You can resize the window as you want, the canvas should cover the viewport without any scroll bar or overflow.

## Handle pixel ratio

Some of you might see a kind of blurry render and artifacts shaped like stairs on the edges (called aliasing), but not all of you. If you do, it's because you are testing on a screen with a pixel ratio greater than **1**.

The pixel ratio corresponds to how many physical pixels you have on the screen for one pixel unit on the software part.

### Some history

A few years ago, all screens had a pixel ratio of **1** and everything was just fine. But when you looked closely at your screen, you could see those pixels, and it was a limitation for how precise images and how thin fonts could be.

The company that did the most about that was Apple. Apple saw an opportunity and started constructing screens with a pixel ratio of **2** called retina. Now, a lot of constructors are doing it, and you can see screens with even higher pixel ratios.

While this is good for the image quality, a pixel ratio of **2** means 4 times more pixels to render. And a pixel ratio of **3** means 9 times more pixels to render.

And guess what? The highest pixel ratios are usually on the weakest devices —mobile ones.

There goes your frame rate.

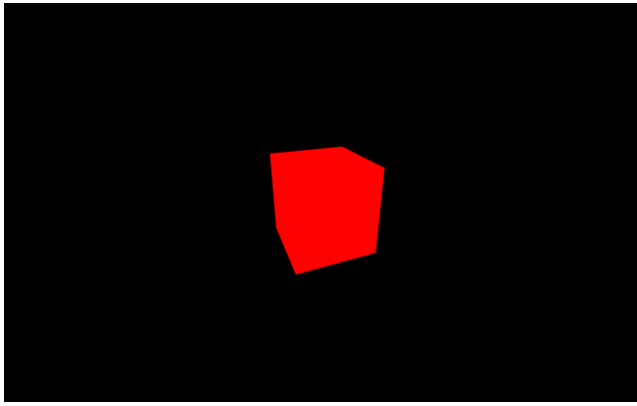
## Handle the pixel ratio

To get the screen pixel ratio you can use **window.devicePixelRatio**, and to update the pixel ratio of your renderer, you simply need to call the **renderer.setPixelRatio(...)**

You might be tempted to simply send the device pixel ratio to that method, but you'll end up with performance issues on high pixel ratio devices.

Having a pixel ratio greater than **2** is mostly marketing. Your eyes will see almost no difference between **2** and **3** but it will create performance issues and empty the battery faster. What you can do is limit the pixel ratio to **2**. To do that, you can use **Math.min()**:

```
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
```



There are techniques to be notified when the pixel ratio changes, but it only concerns users having multiple screens with different pixel ratios and they usually resize their window when changing from a screen to another. That is why we will simply add this method to the **resize** callback too:

```
window.addEventListener('resize', () =>
{
  // Update sizes
  sizes.width = window.innerWidth
  sizes.height = window.innerHeight

  // Update camera
  camera.aspect = sizes.width / sizes.height

  // Update renderer
  renderer.setSize(sizes.width, sizes.height)
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
})
```

## Handle fullscreen

Now that we have the canvas taking all the available space with the right pixel ratio, it's time to add support to the fullscreen.

First, we need to decide what action will trigger the fullscreen mode. It could be an HTML button, but instead, we're going to use a double click.

When the double click happens, we will toggle the fullscreen —meaning that if the window is not in fullscreen, a double-click will enable fullscreen mode, and if the window is already in fullscreen, a double-click will exit fullscreen mode.

First, we need to listen to the double click event, and we can do that with the **dblclick** event:

```
window.addEventListener('dblclick', () =>
{
  console.log('double click')
})
```

This event will work on most modern browsers except for Chrome Android: [https://developer.mozilla.org/docs/Web/API/Element/dblclick\\_event](https://developer.mozilla.org/docs/Web/API/Element/dblclick_event)

Now that we have our event, we need 3 things:

- A way to know if it's already in fullscreen
- A method to go to the fullscreen mode

- A method to leave the fullscreen mode

To know if we are already in fullscreen or not, we can use **document.fullscreenElement**:

```
window.addEventListener('dblclick', () =>
{
  if(!document.fullscreenElement)
  {
    console.log('go fullscreen')
  }
  else
  {
    console.log('leave fullscreen')
  }
})
```

The method to request the fullscreen is associated with the element. It's because you can choose what will be in fullscreen. It can be the whole page, any DOM element or the **<canvas>**.

We will use the **<canvas>** and call the **requestFullscreen()** method on it:

```
window.addEventListener('dblclick', () =>
{
  if(!document.fullscreenElement)
  {
    canvas.requestFullscreen()
  }
  else
  {
    console.log('leave fullscreen')
  }
})
```

The method to leave the fullscreen mode is available directly on the **document**:

```
window.addEventListener('dblclick', () =>
{
  if(!document.fullscreenElement)
  {
    canvas.requestFullscreen()
  }
  else
  {
    document.exitFullscreen()
  }
})
```

You can test the result by double-clicking anywhere to toggle the fullscreen mode. Unfortunately, this won't work on Safari.

This browser is taking its time to support officially simple features like the fullscreen, and we need to use prefixed versions to make it work for **document.fullscreenElement**, **canvas.requestFullscreen**, and **document.exitFullscreen**:

```
window.addEventListener('dblclick', () =>
{
  const fullscreenElement = document.fullscreenElement ||
document.webkitFullscreenElement

  if(!fullscreenElement)
  {
    if(canvas.requestFullscreen)
    {
      canvas.requestFullscreen()
    }
    else if(canvas.webkitRequestFullscreen)
    {
      canvas.webkitRequestFullscreen()
    }
  }
  else
  {
    if(document.exitFullscreen)
    {
      document.exitFullscreen()
    }
    else if(document.webkitExitFullscreen)
    {
      document.webkitExitFullscreen()
    }
  }
})
```

Everything should work fine on all modern browsers.

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 7) GEOMETRIES

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

---

### Introduction

Until now, we only used the [BoxGeometry](#) class to create our cube. In this lesson, we will discover various other geometries, but first, we need to understand what geometry really is.

### What is a geometry?

In Three.js, geometries comprise vertices (point coordinates in 3D spaces) and faces (triangles that join those vertices to create a surface).

We use geometries to create meshes, but you can also use geometries to form particles. Each vertex (singular of vertices) will correspond to a particle, but this is for a future lesson.

We can store more data than the position in the vertices. A good example would be to talk about the UV coordinates or the normals. As you'll see, we will learn more about those later.

### The different built-in geometries

Three.js has many built-in geometries. While you don't need to know precisely how to instantiate each, it is good to know they exist.

We will see all the built-in geometries inherited from the [BufferGeometry](#) class. This class has many built-in methods like **translate(...)**, **rotateX(...)**, **normalize()**, etc., but we are not going to use them in this lesson.

Most of the Geometries documentation pages have examples.

- [BoxGeometry](#) To create a box.
- [PlaneGeometry](#) To create a rectangle plane.
- [CircleGeometry](#) To create a disc or a portion of a disc (like a pie chart).
- [ConeGeometry](#) To create a cone or a portion of a cone. You can open or close the base of the cone.
- [CylinderGeometry](#) To create a cylinder. You can open or close the ends of the cylinder and you can change the radius of each end.
- [RingGeometry](#) To create a flat ring or portion of a flat circle.
- [TorusGeometry](#) To create a ring that has a thickness (like a donut) or portion of a ring.
- [TorusKnotGeometry](#) To create some sort of knot geometry.
- [DodecahedronGeometry](#) To create a 12 faces sphere. You can add details for a rounder sphere.
- [OctahedronGeometry](#) To create a 8 faces sphere. You can add details for a rounder sphere.
- [TetrahedronGeometry](#) To create a 4 faces sphere (it won't be much of a sphere if you don't increase details). You can add details for a rounder sphere.
- [IcosahedronGeometry](#) To create a sphere composed of triangles that have roughly the same size.

- [SphereGeometry](#) To create the most popular type of sphere where faces looks like quads (quads are just a combination of two triangles).
- [ShapeGeometry](#) To create a shape based on a path.
- [TubeGeometry](#) To create a tube following a path.
- [ExtrudeGeometry](#) To create an extrusion based on a path. You can add and control the bevel.
- [LatheGeometry](#) To create a vase or portion of a vase (more like a revolution).
- [TextGeometry](#) To create a 3D text. You'll have to provide the font in typeface json format.

If you need a particular geometry not supported by Three.js, you can create your geometry in JavaScript, or you can make it in a 3D software, export it and import it into your project. We will learn more about that later.

### Box example

We already made a cube but didn't discuss the parameters much. Most geometries have parameters, and you should always look at the documentation before using it.

The [BoxGeometry](#) has 6 parameters:

- **width**: The size on the **x** axis
- **height**: The size on the **y** axis
- **depth**: The size on the **z** axis
- **widthSegments**: How many subdivisions in the **x** axis
- **heightSegments**: How many subdivisions in the **y** axis
- **depthSegments**: How many subdivisions in the **z** axis

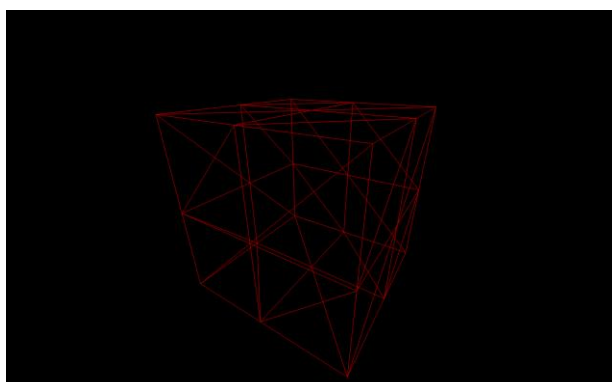
Subdivisions correspond to how many triangles should compose the face. By default, it's **1**, meaning there will only be 2 triangles per face. If you set the subdivision to **2**, you'll end up with 8 triangles per face:

```
const geometry = new THREE.BoxGeometry(1, 1, 1, 2, 2, 2)
```

The problem is that we cannot see these triangles.

A good solution is to add **wireframe: true** to our material. The wireframe will show the lines that delimit each triangle:

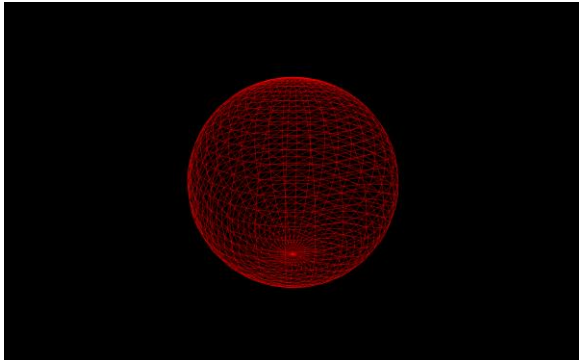
```
const material = new THREE.MeshBasicMaterial({ color: 0xff0000, wireframe: true })
```



As you can see, there are 8 triangles by face.

While this is not relevant for a flat face cube, it gets more interesting when using a [SphereGeometry](#):

```
const geometry = new THREE.SphereGeometry(1, 32, 32)
```



The more subdivisions we add, the less we can distinguish the faces. But keep in mind that too many vertices and faces will affect performances.

### Creating your buffer geometry

Sometimes, we need to create our geometries. If the geometry is very complex or with a precise shape, it's better to create it in 3D software (and we will cover that in a future lesson), but if the geometry isn't too complex, we can build it ourselves by using [BufferGeometry](#).

To create your buffer geometry, start by instantiating an empty [BufferGeometry](#). We will create a simple triangle:

```
// Create an empty BufferGeometry
const geometry = new THREE.BufferGeometry()
```

To add vertices to a [BufferGeometry](#), you must start with a [Float32Array](#).

[Float32Array](#) is a native JavaScript-typed array. You can only store floats inside, and the length of that array is fixed.

To create a [Float32Array](#), you can specify its length and then fill it later:

```
const positionsArray = new Float32Array(9)

// First vertice
positionsArray[0] = 0
positionsArray[1] = 0
positionsArray[2] = 0

// Second vertice
positionsArray[3] = 0
positionsArray[4] = 1
positionsArray[5] = 0

// Third vertice
positionsArray[6] = 1
positionsArray[7] = 0
positionsArray[8] = 0
```

Or you can pass an array:

```
const positionsArray = new Float32Array([
  0, 0, 0, // First vertex
  0, 1, 0, // Second vertex
  1, 0, 0  // Third vertex
])
```

As you can see, the coordinates of the vertices are specified linearly. The array is a one-dimensional array where you specify the **x**, **y**, and **z** of the first vertex, followed by the **x**, **y**, and **z** of the second vertex, and so on.

Before you can send that array to the [BufferGeometry](#), you have to transform it into a [BufferAttribute](#).

The first parameter corresponds to your typed array and the second parameter corresponds to how much values make one vertex attribute. As we saw earlier, to read this array, we have to go 3 by 3 because a vertex position is composed of 3 values (**x**, **y** and **z**):

```
const positionsAttribute = new THREE.BufferAttribute(positionsArray, 3)
```

Then we can add this attribute to our [BufferGeometry](#) using the **setAttribute(...)** method. The first parameter is the name of this attribute and the second parameter is the value:

```
geometry.setAttribute('position', positionsAttribute)
```

We chose '**position**' as the name because Three.js internal shaders will look for that value to position the vertices. We will see more about that in the shaders lessons.

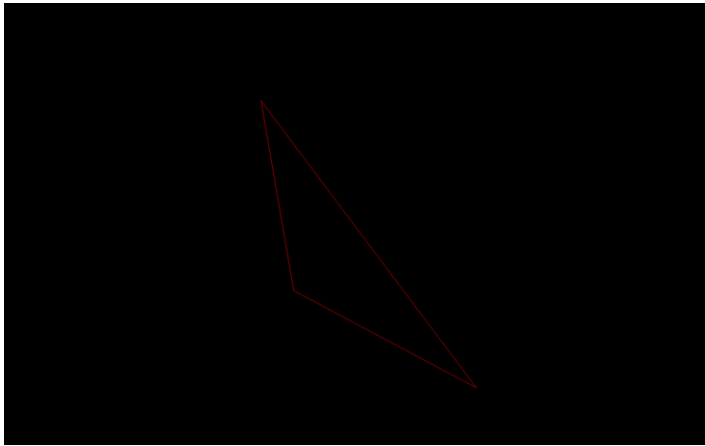
The faces will be automatically created following the order of the vertices.

All together:

```
// Create an empty BufferGeometry
const geometry = new THREE.BufferGeometry()

// Create a Float32Array containing the vertices position (3 by 3)
const positionsArray = new Float32Array([
  0, 0, 0, // First vertex
  0, 1, 0, // Second vertex
  1, 0, 0  // Third vertex
])

// Create the attribute and name it 'position'
const positionsAttribute = new THREE.BufferAttribute(positionsArray, 3)
geometry.setAttribute('position', positionsAttribute)
```

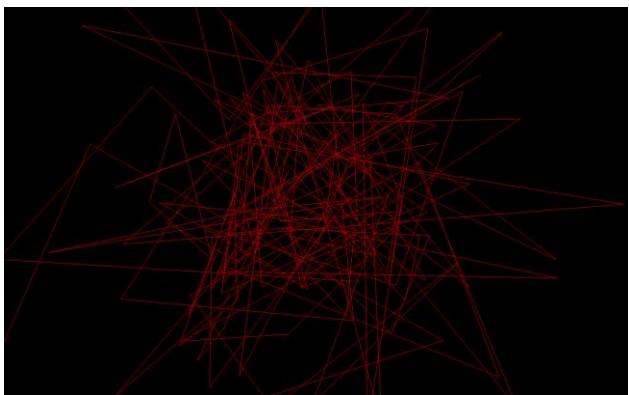


We can also create a bunch of random triangles:

```
// Create an empty BufferGeometry
const geometry = new THREE.BufferGeometry()

// Create 50 triangles (450 values)
const count = 50
const positionsArray = new Float32Array(count * 3 * 3)
for(let i = 0; i < count * 3 * 3; i++)
{
    positionsArray[i] = (Math.random() - 0.5) * 4
}

// Create the attribute and name it 'position'
const positionsAttribute = new THREE.BufferAttribute(positionsArray, 3)
geometry.setAttribute('position', positionsAttribute)
```



The only difficulty might be the **count \* 3 \* 3** part but it's quite simple to explain: We need **50** triangles. Each triangle is composed of **3** vertices and each vertex is composed of **3** values (**x**, **y**, and **z**).

## Index

One interesting thing with [BufferGeometry](#) is that you can mutualize vertices using the **index** property. Consider a cube. Multiple faces can use some vertices like the ones in the corners. And if you look closely, every vertex can be used by various neighbor triangles. That will result in a smaller attribute array and performance improvement. But we won't cover this part in that lesson.

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 8) DEBUG UI

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

---

### Introduction

An essential aspect of every creative project is making debugging easy and tweaking your code. The developer (you) and other actors working on the project (like designers or even the client) must be able to change as many parameters as possible.

You must consider this for them to find the perfect color, speed, quantity, etc., for the best experience. You might even get unexpected results that look great.

First, we need a debug UI.

While you can create your own debug UI using HTML / CSS / JS, there are already multiple libraries:

- [dat.GUI](#)
- [control-panel](#)
- [ControlKit](#)
- [Uil](#)
- [Tweakpane](#)
- [Guify](#)
- [Oui](#)

All of these can do what we want, but we will use the most popular one, which is [dat.GUI](#). Feel free to try the other ones.

### Dat.GUI vulnerabilities

Dat.GUI hasn't been updated for a long time, and some vulnerability warnings might show up if we add the library to our project.

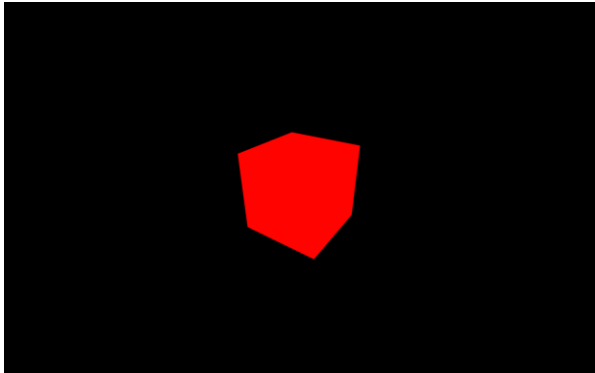
Fortunately, an alternative library named **lil-gui** can be used as a "drop-in replacement for dat.gui". We can use it the same way we would have used dat.gui.

The course has been mostly written and recorded with dat.gui, and the texts, screenshots, and videos will refer to dat.gui, but you should install and use lil-gui instead (which is what we will do right below).

The next lesson's starter files will be using lil-gui.

## Setup

We have our cube in the starter, but the dependencies don't include Dat.GUI. We will add it and create some tweaks.



## How to implement Dat.GUI

To add Dat.GUI to our project, we can use the dependency manager provided with Node.js called NPM (just like we did for GSAP in a previous lesson).

In your terminal (while the server is not running or by using another terminal window on the same folder), run **npm install --save lil-gui**

As mentioned earlier, we are installing lil-gui instead of dat.gui, but we will refer to it as dat.gui in the rest of the course.

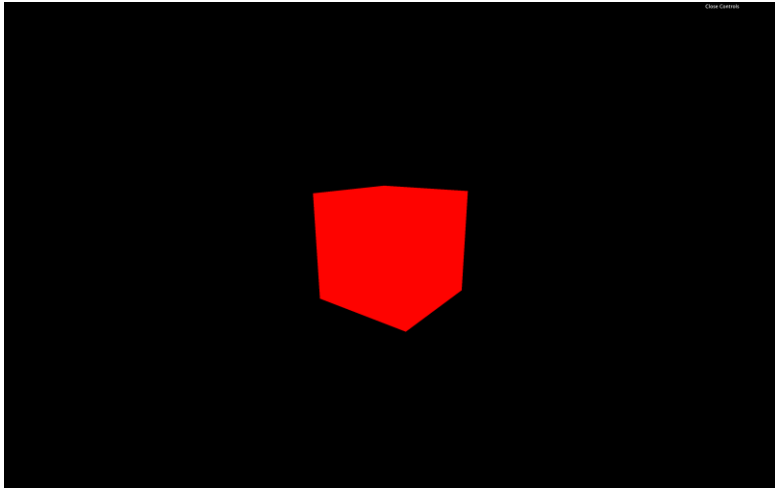
Dat.GUI is now available in the **/node\_modules/** folder, which we can import in our **script.js**. Don't forget to relaunch the server:

```
import './style.css'
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import gsap from 'gsap'
import * as dat from 'lil-gui'

// ...
```

You can now instantiate Dat.GUI:

```
/**
 * Debug
 */
const gui = new dat.GUI()
```



This will result in an empty panel on the top right corner of the screen.

There are different types of elements you can add to that panel:

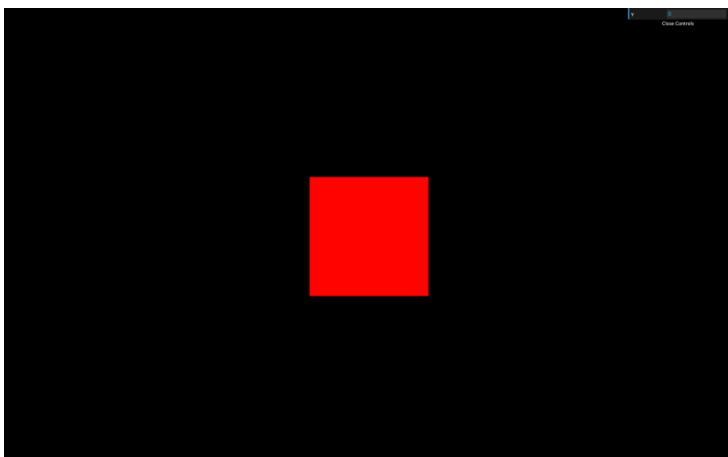
- **Range** —for numbers with minimum and maximum value
- **Color** —for colors with various formats
- **Text** —for simple texts
- **Checkbox** —for booleans (**true** or **false**)
- **Select** —for a choice from a list of values
- **Button** —to trigger functions
- **Folder** —to organize your panel if you have too many elements

Let's see some of those.

## Add elements

To add an element to the panel, you must use **gui.add(...)**. The first parameter is an object and the second parameter is the property of that object you want to tweak. You need to set it after you created the concerned object:

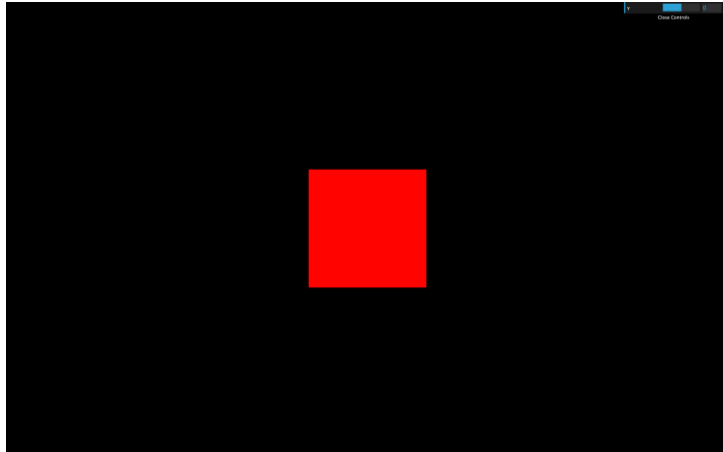
```
gui.add(mesh.position, 'y')
```



A range should appears in the panel. Try to change it and watch the cube moving accordingly.

To specify the minimum value, the maximum value and the precision, you can set them in the parameters:

```
gui.add(mesh.position, 'y', - 3, 3, 0.01)
```



Or you can use the methods **min(...)**, **max(...)** and **step(...)** by chaining directly after the **add(...)** method:

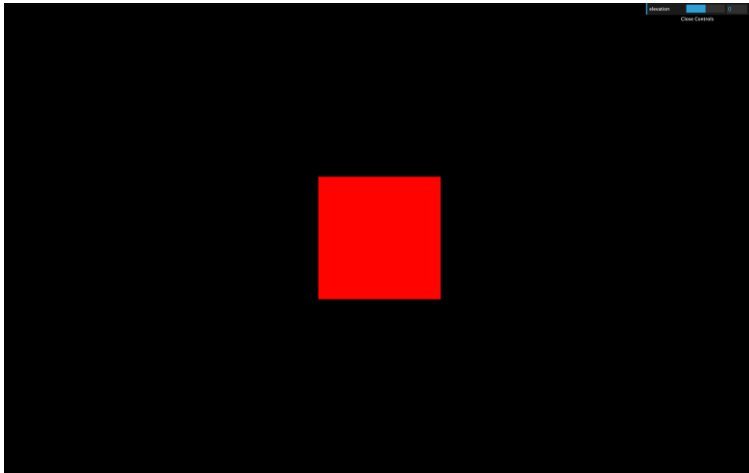
```
gui.add(mesh.position, 'y').min(- 3).max(3).step(0.01)
```

If you don't like having too many methods chained in one line, you can simply add line breaks:

```
gui
    .add(mesh.position, 'y')
    .min(- 3)
    .max(3)
    .step(0.01)
```

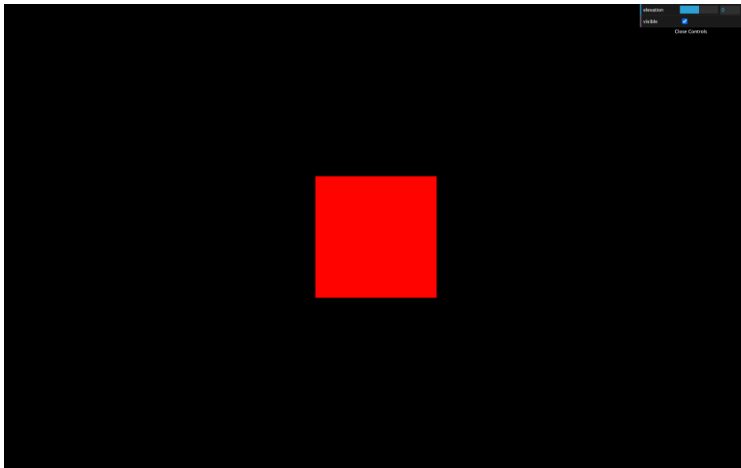
To change the label, use the **name(...)** method:

```
gui
    .add(mesh.position, 'y')
    .min(- 3)
    .max(3)
    .step(0.01)
    .name('elevation')
```



Dat.GUI will automatically detect what kind of property you want to tweak and use the corresponding element. A good example is the **visible** property of [Object3D](#). It is a boolean that, if **false**, will hide the object:

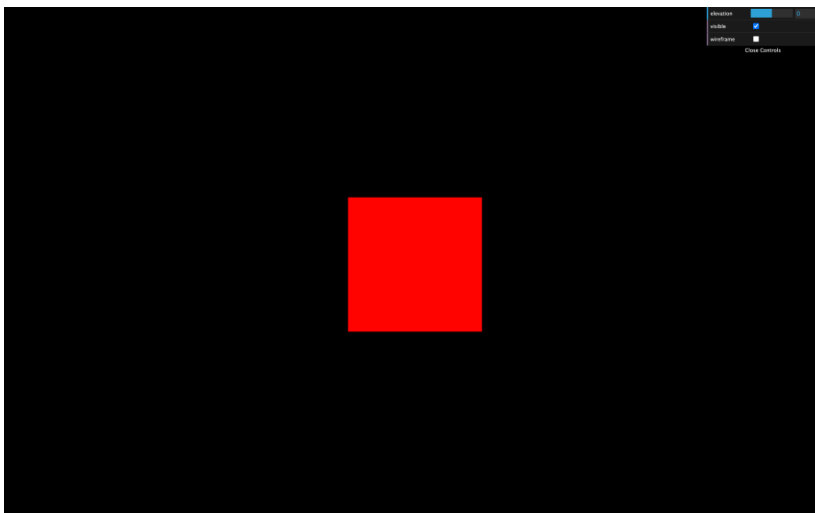
```
gui.add(mesh, 'visible')
```



As you can see, Dat.GUI chose a checkbox because the **visible** property is a boolean.

We can do the same with the **wireframe** property of the material:

```
gui.add(material, 'wireframe')
```



## Colors

Handling colors is a little harder.

First, we need to use **addColor(...)** instead of **add(...)**. This is due to Dat.GUI not being able to know if you want to tweak a text, a number or a color just by the type of the property.

Secondly, you'll have to create an intermediate object with the color in its properties and use that property in your material. That is due to the Three.js material not having a clean and accessible value like **#ff0000**.

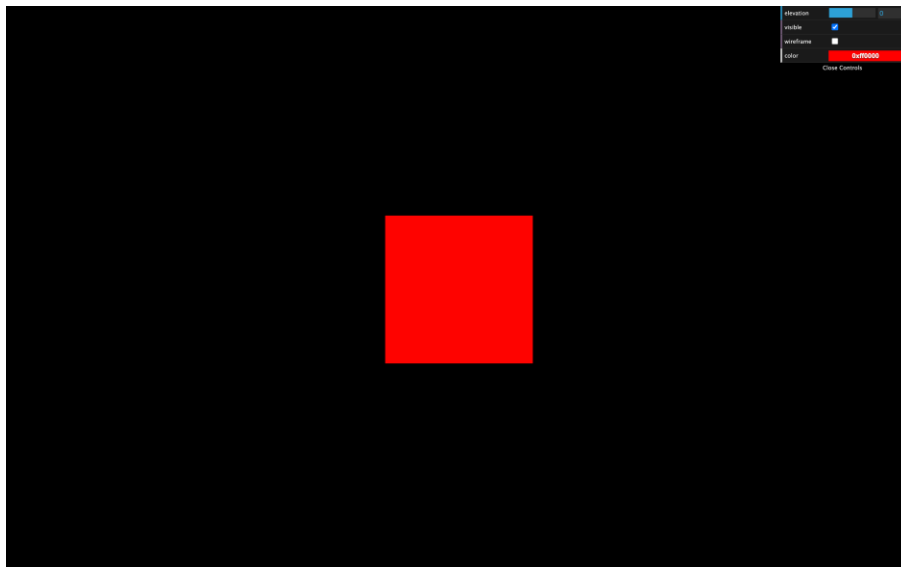
Actually, because we are using lil-gui instead of Dat.GUI, we can use **addColor(...)** directly on the material. But since the technique we are going to see can be used in other cases, we are going to see it anyway.

Create a **parameter** variable at the start of your code right after the **import** part:

```
const parameters = {  
  color: 0xff0000  
}
```

Then, after instantiating your **gui** variable, add the following tweak:

```
gui.addColor(parameters, 'color')
```



You should see a color picker in your panel. The problem is that changing this color doesn't affect the material. It does change the **color** property of the **parameter** variable, but we don't even use that variable in our material.

To fix that, we need Dat.GUI to alert us when the value changed. We can do that by chaining the **onChange(...)** method and updating the material color using **material.color.set(...)**. This method is very useful because of how many formats you can use like **'#ff0000'**, **'#f00'**, **0xff0000** or even **'red'**:

```
const parameters = {  
  color: 0xff0000  
}  
  
// ...  
  
gui  
  .addColor(parameters, 'color')
```

```
.onChange(() =>
{
    material.color.set(parameters.color)
})
```

Currently, the **0xff0000** color is specified at two places: in the **parameters** object and in the **material**.

While it's not a big deal, if we want to change the color, we'll have to do it at those two places.

Let's simply fix that by using the **parameters.color** property in our material:

```
const material = new THREE.MeshBasicMaterial({ color: parameters.color })
```

## Functions

To trigger a function, like the color value, we must add that function to an object. We can use the **parameters** object we created earlier to add a **spin** property containing the function that will animate the cube:

```
const parameters = {
    color: 0xff0000,
    spin: () =>
    {
        gsap.to(mesh.rotation, { duration: 1, y: mesh.rotation.y + Math.PI *
2 })
    }
}
```

And again, we can add the tweak to our **gui** after instantiating it:

```
gui.add(parameters, 'spin')
```

You should see a **spin** button, and clicking on it will result on your cube doing a 360 rotation.

## How and when to use it

We will use our debug panel at specific moments of the next exercises. But feel free to add as many tweaks as you want. It's an excellent way to practice and start building some creative stuff.

I would recommend you add tweaks as you progress. If you think of adding all the tweaks at the end of the project, you'll end up with no tweaks.

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 9) TEXTURES

**COURSE/SUBJECT:** *APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES*

**COURSE NO.:** *ITE-18*

**INSTRUCTOR/PROF:** *JAMES EARL D. CUBILLAS, M.Sc., ECT*

---

### Introduction

Bored with your red cube yet? It's time to add some textures.

But first, what are textures, and what can we do with them?

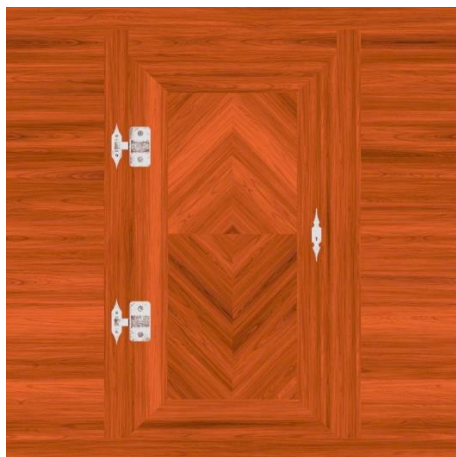
### What are textures?

As you probably know, Textures are images that will cover the surface of your geometries. Many types of textures can have different effects on the appearance of your geometry. It's not just about the color.

Here are the most common types of textures using a famous [door texture](#) by João Paulo. Buy him a [Ko-fi](#) or become a [Patreon](#) if you like his work.

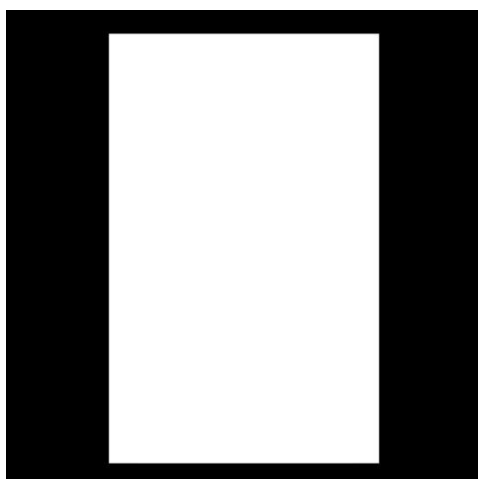
### Color (or albedo)

The albedo texture is the most simple one. It'll only take the pixels of the texture and apply them to the geometry.



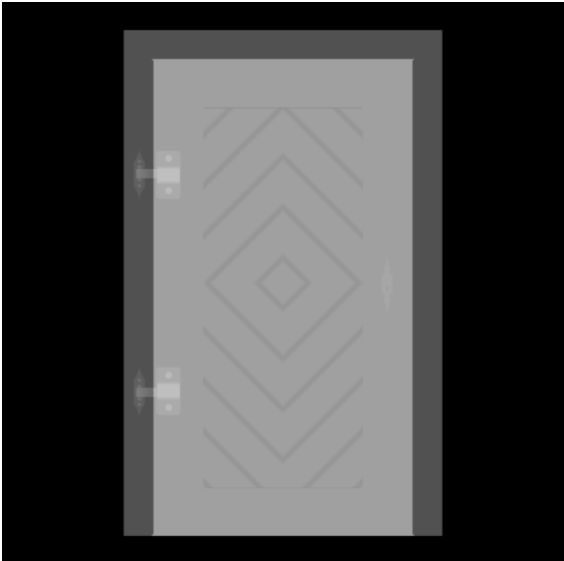
### Alpha

The alpha texture is a grayscale image where white will be visible, and black won't.



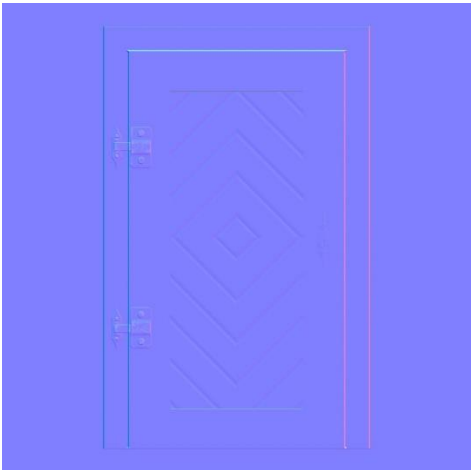
**Height**

The height texture is a grayscale image that will move the vertices to create some relief. You'll need to add subdivision if you want to see it.



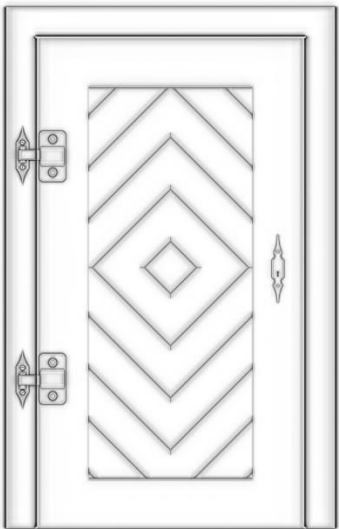
**Normal**

The normal texture will add small details. It won't move the vertices, but it will lure the light into thinking that the face is oriented differently. Normal textures are very useful to add details with good performance because you don't need to subdivide the geometry.



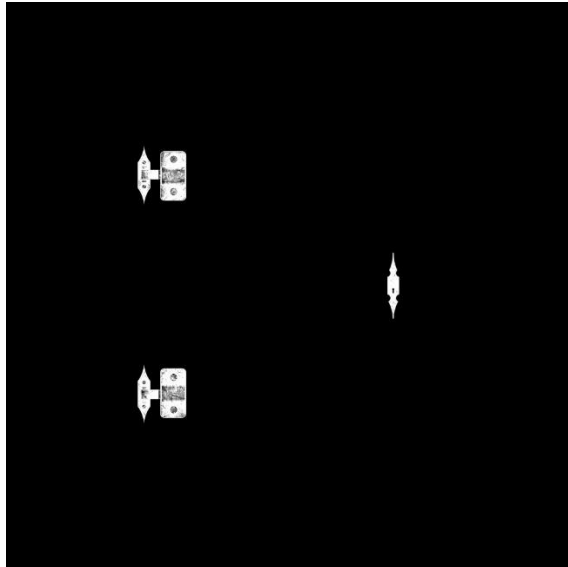
**Ambient occlusion**

The ambient occlusion texture is a grayscale image that will fake shadow in the surface's crevices. While it's not physically accurate, it certainly helps to create contrast.



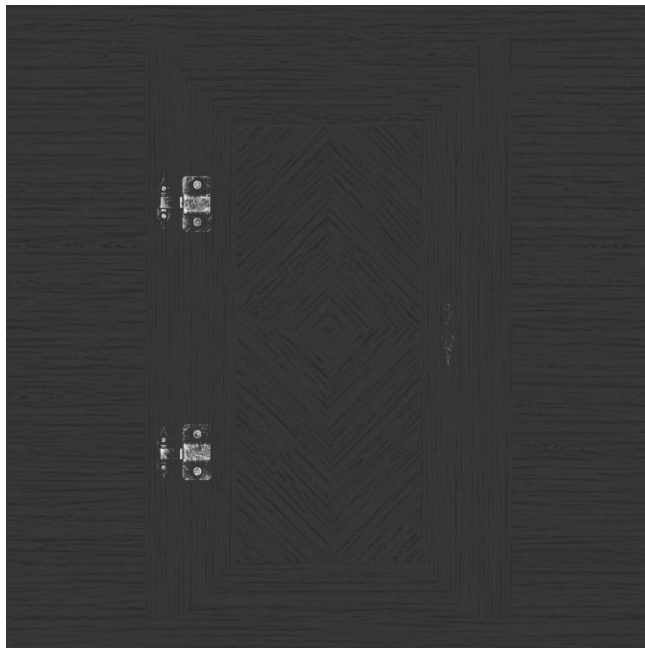
## Metalness

The metalness texture is a grayscale image that will specify which part is metallic (white) and non-metallic (black). This information will help to create reflection.



## Roughness

The roughness is a grayscale image that comes with metalness, and that will specify which part is rough (white) and which part is smooth (black). This information will help to dissipate the light. A carpet is very rugged, and you won't see the light reflection on it, while the water's surface is very smooth, and you can see the light reflecting on it. Here, the wood is uniform because there is a clear coat on it.



## PBR – Physically Based Rendering

Those textures (especially the metalness and the roughness) follow what we call PBR principles. PBR stands for Physically Based Rendering. It regroups many techniques that tend to follow real-life directions to get realistic results.

While there are many other techniques, PBR is becoming the standard for realistic renders, and many software, engines, and libraries are using it.

For now, we will simply focus on how to load textures, how to use them, what transformations we can apply, and how to optimize them. We will see more about PBR in later lessons, but if you're curious, you can learn more about it here:

- <https://marmoset.co/posts/basic-theory-of-physically-based-rendering/>
- <https://marmoset.co/posts/physically-based-rendering-and-you-can-too/>

## How to load textures

### Getting the URL of the image

To load the texture, we need the URL of the image file.

Because we are using a build tool, there are two ways of getting it.

You can put the image texture in the **/src/** folder and import it like you would import a JavaScript dependency:

```
import imageSource from './image.png'

console.log(imageSource)
```

Or you can put that image in the **/static/** folder and access it just by adding the path of the image (without **/static**) to the URL:

```
const imageSource = '/image.png'

console.log(imageSource)
```

Be careful, this **/static/** folder only works because of the Vite template's configuration. If you are using other types of bundler, you might need to adapt your project.

We will use the **/static/** folder technique for the rest of the course.

### Loading the image

You can find the door textures we just saw in the **/static/** folder, and there are multiple ways of loading them.

#### Using native JavaScript

With native JavaScript, first, you must create an **Image** instance, listen to the **load** event, and then change its **src** property to start loading the image:

```
const image = new Image()
image.onload = () => {
  console.log('image loaded')
}
image.src = '/textures/door/color.jpg'
```

You should see **'image loaded'** appears in the console. As you can see, we set the source to **'/textures/door/color.jpg'** without the **/static** folder in the path.

We cannot use that image directly. We need to create a [Texture](#) from that image first.

This is because WebGL needs a very specific format that can be access by the GPU and also because some changes will be applied to the textures like the mipmapping but we will see more about that a little later.

Create the texture with the [Texture](#) class:

```
const image = new Image()
image.addEventListener('load', () =>
{
    const texture = new THREE.Texture(image)
})
image.src = '/textures/door/color.jpg'
```

What we need to do now is use that **texture** in the **material**. Unfortunately, the **texture** variable has been declared in a function and we can not access it outside of this function. This is a JavaScript limitation called scope.

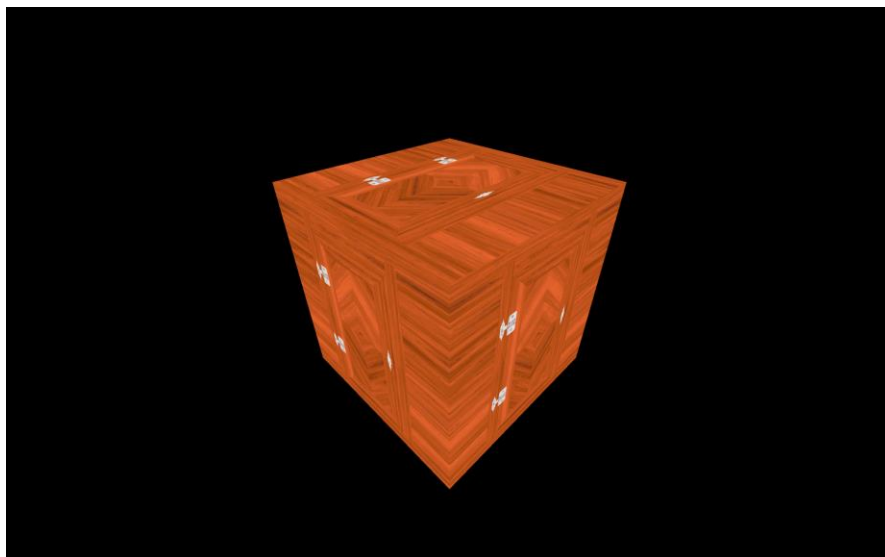
We could create the mesh inside the function, but there is a better solution consisting on creating the texture outside of the function and then updating it once the image is loaded by setting the texture **needsUpdate** property to **true**:

```
const image = new Image()
const texture = new THREE.Texture(image)
image.addEventListener('load', () =>
{
    texture.needsUpdate = true
})
image.src = '/textures/door/color.jpg'
```

While doing this, you can immediately use the **texture** variable immediately, and the image will be transparent until it is loaded.

To see the texture on the cube, replace the **color** property by **map** and use the **texture** as value:

```
const material = new THREE.MeshBasicMaterial({ map: texture })
```



You should see the door texture on each side of your cube.

## Using TextureLoader

The native JavaScript technique is not that complicated, but there is an even more straightforward way with [TextureLoader](#).

Instantiate a variable using the [TextureLoader](#) class and use its **.load(...)** method to create a texture:

```
const textureLoader = new THREE.TextureLoader()
const texture = textureLoader.load('/textures/door/color.jpg')
```

Internally, Three.js will do what it did before to load the image and update the texture once it's ready.

You can load as many textures as you want with only one [TextureLoader](#) instance.

You can send 3 functions after the path. They will be called for the following events:

- **load** when the image loaded successfully
- **progress** when the loading is progressing
- **error** if something went wrong

```
const textureLoader = new THREE.TextureLoader()
const texture = textureLoader.load(
  '/textures/door/color.jpg',
  () => {
    console.log('loading finished')
  },
  () => {
    console.log('loading progressing')
  },
  () => {
    console.log('loading error')
  }
)
```

If the texture doesn't work, it might be useful to add those callback functions to see what is happening and spot errors.

## Using the LoadingManager

Finally, if you have multiple images to load and want to mutualize the events like being notified when all the images are loaded, you can use a [LoadingManager](#).

Create an instance of the [LoadingManager](#) class and pass it to the [TextureLoader](#):

```
const loadingManager = new THREE.LoadingManager()
const textureLoader = new THREE.TextureLoader(loadingManager)
```

You can listen to the various events by replacing the following properties by your own functions **onStart**, **onLoad**, **onProgress**, and **onError**:

```
const loadingManager = new THREE.LoadingManager()
loadingManager.onStart = () =>
{
    console.log('loading started')
}
loadingManager.onLoad = () =>
{
    console.log('loading finished')
}
loadingManager.onProgress = () =>
{
    console.log('loading progressing')
}
loadingManager.onError = () =>
{
    console.log('loading error')
}

const textureLoader = new THREE.TextureLoader(loadingManager)
```

You can now start loading all the images you need:

```
// ...

const colorTexture = textureLoader.load('/textures/door/color.jpg')
const alphaTexture = textureLoader.load('/textures/door/alpha.jpg')
const heightTexture = textureLoader.load('/textures/door/height.jpg')
const normalTexture = textureLoader.load('/textures/door/normal.jpg')
const ambientOcclusionTexture =
textureLoader.load('/textures/door/ambientOcclusion.jpg')
const metalnessTexture = textureLoader.load('/textures/door/metalness.jpg')
const roughnessTexture = textureLoader.load('/textures/door/roughness.jpg')
```

As you can see here, we renamed the **texture** variable **colorTexture** so don't forget to change it in the **material** too:

```
const material = new THREE.MeshBasicMaterial({ map: colorTexture })
```

The [LoadingManager](#) is very useful if you want to show a loader and hide it only when all the assets are loaded. As we will see in a future lesson, you can also use it with other types of loaders.

## UV unwrapping

While it is quite logical how to place a texture on a cube, things can be a little trickier for other geometries.

Try to replace your BoxGeometry with other geometries:

```
const geometry = new THREE.BoxGeometry(1, 1, 1)

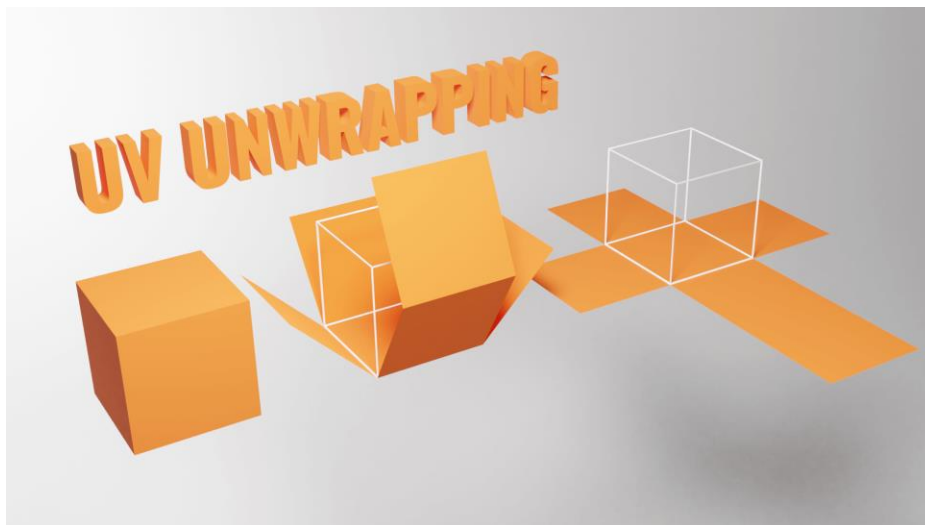
// Or
const geometry = new THREE.SphereGeometry(1, 32, 32)

// Or
const geometry = new THREE.ConeGeometry(1, 1, 32)

// Or
const geometry = new THREE.TorusGeometry(1, 0.35, 32, 100)
```

As you can see, the texture is being stretched or squeezed in different ways to cover the geometry.

That is called **UV unwrapping**. You can imagine that like unwrapping an origami or a candy wrap to make it flat. Each vertex will have a 2D coordinate on a flat (usually square) plane.

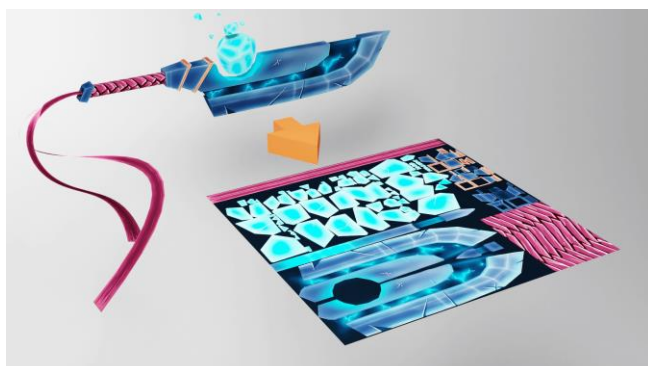


You can actually see those UV 2D coordinates in the **geometry.attributes.uv** property:

```
console.log(geometry.attributes.uv)
```

Those UV coordinates are generated by Three.js when you use the primitives. If you create your own geometry and want to apply a texture to it, you'll have to specify the UV coordinates.

If you are making the geometry using a 3D software, you'll also have to do the UV unwrapping.



Don't worry; most 3D software also has auto unwrapping that should do the trick.

## Transforming the texture

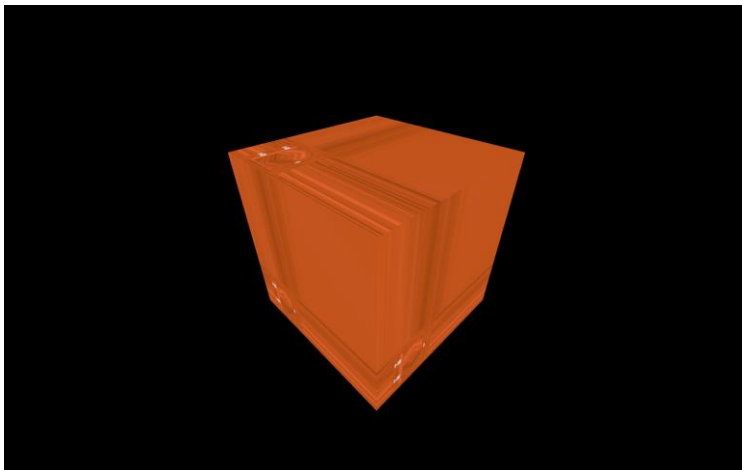
Let's get back to our cube with one texture and see what kind of transformations we can apply to that texture.

### Repeat

You can repeat the texture using the **repeat** property, which is a [Vector2](#), meaning that it has **x** and **y** properties.

Try to change these properties:

```
const colorTexture = textureLoader.load('/textures/door/color.jpg')
colorTexture.repeat.x = 2
colorTexture.repeat.y = 3
```

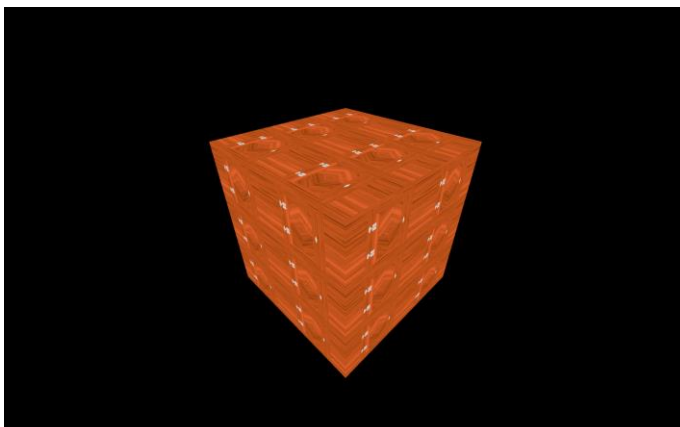


As you can see, the texture is not repeating, but it is smaller, and the last pixel seems stretched.

That is due to the texture not being set up to repeat itself by default. To change that, you have to update the **wrapS** and **wrapT** properties using the **THREE.RepeatWrapping** constant.

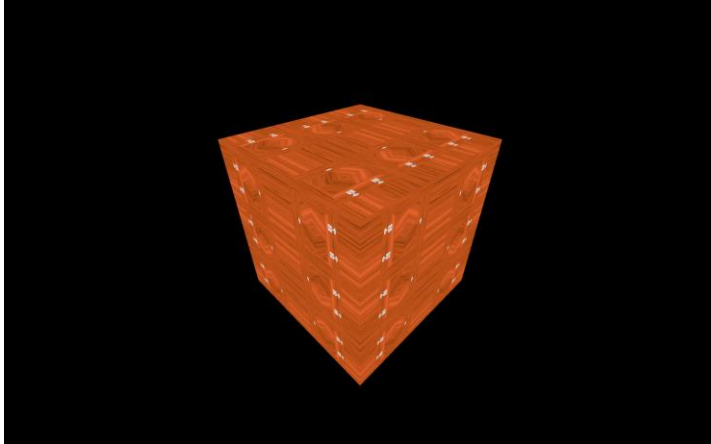
- **wrapS** is for the **x** axis
- **wrapT** is for the **y** axis

```
colorTexture.wrapS = THREE.RepeatWrapping
colorTexture.wrapT = THREE.RepeatWrapping
```



You can also alternate the direction with **THREE.MirroredRepeatWrapping**:

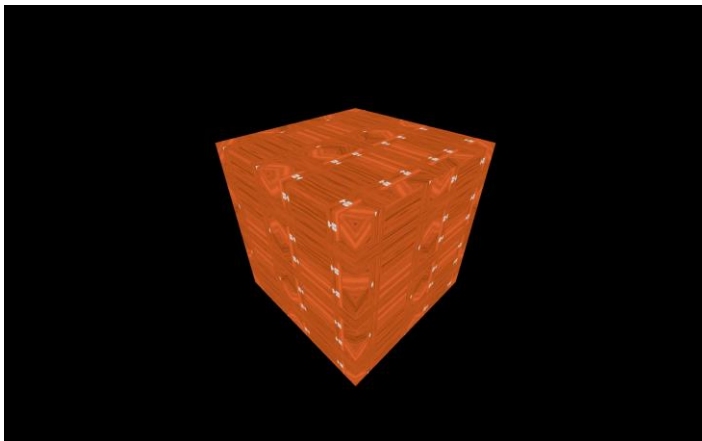
```
colorTexture.wrapS = THREE.MirroredRepeatWrapping  
colorTexture.wrapT = THREE.MirroredRepeatWrapping
```



### Offset

You can offset the texture using the **offset** property that is also a [Vector2](#) with **x** and **y** properties. Changing these will simply offset the UV coordinates:

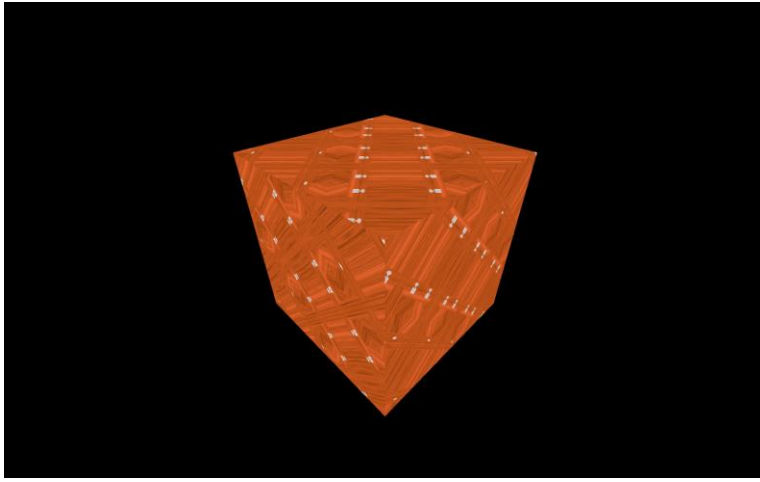
```
colorTexture.offset.x = 0.5  
colorTexture.offset.y = 0.5
```



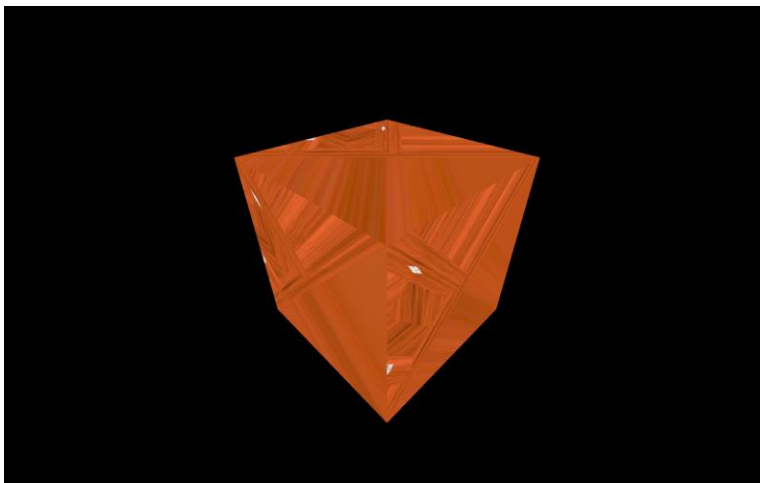
### Rotation

You can rotate the texture using the **rotation** property, which is a simple number corresponding to the angle in radians:

```
colorTexture.rotation = Math.PI * 0.25
```



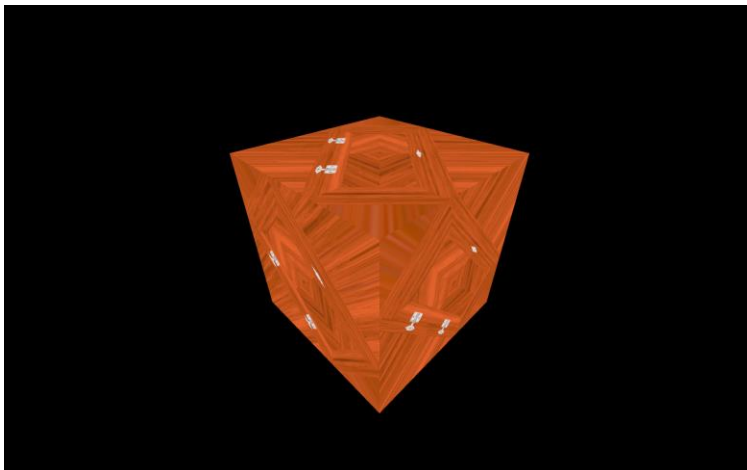
If you remove the **offset** and **repeat** properties, you'll see that the rotation occurs around the bottom left corner of the cube's faces:



That is, in fact, the **0, 0** UV coordinates. If you want to change the pivot of that rotation, you can do it using the **center** property which is also a [Vector2](#):

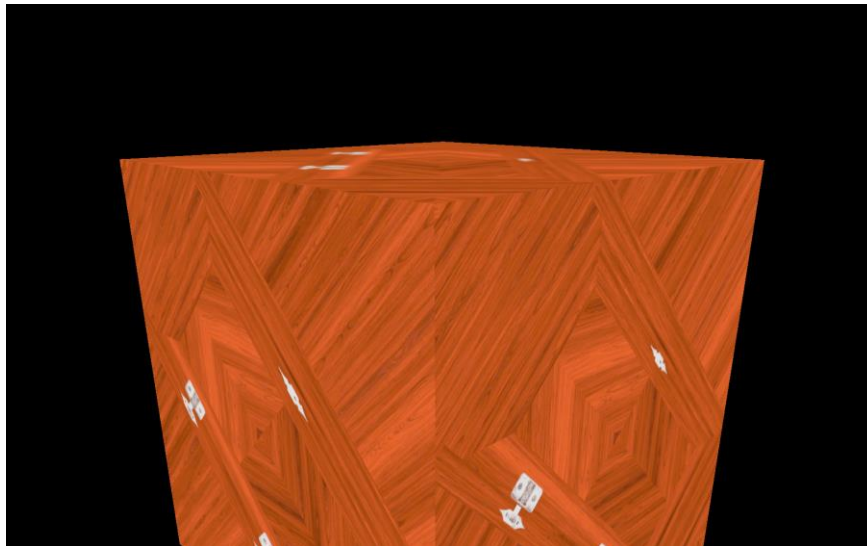
```
colorTexture.rotation = Math.PI * 0.25  
colorTexture.center.x = 0.5  
colorTexture.center.y = 0.5
```

The texture will now rotate on its center.



## Filtering and Mipmapping

If you look at the cube's top face while this face is almost hidden, you'll see a very blurry texture.



That is due to the filtering and the mipmapping.

Mipmapping (or "mip mapping" with a space) is a technique that consists of creating half a smaller version of a texture again and again until you get a 1x1 texture. All those texture variations are sent to the GPU, and the GPU will choose the most appropriate version of the texture.

Three.js and the GPU already handle all of this, and you can just set what filter algorithm to use. There are two types of filter algorithms: the minification filter and the magnification filter.

### Minification filter

The minification filter happens when the pixels of texture are smaller than the pixels of the render. In other words, the texture is too big for the surface, it covers.

You can change the minification filter of the texture using the **minFilter** property.

There are 6 possible values:

- **THREE.NearestFilter**
- **THREE.LinearFilter**
- **THREE.NearestMipmapNearestFilter**
- **THREE.NearestMipmapLinearFilter**
- **THREE.LinearMipmapNearestFilter**
- **THREE.LinearMipmapLinearFilter**

The default is **THREE.LinearMipmapLinearFilter**. If you are not satisfied with how your texture looks, you should try the other filters.

We won't see each one, but we will test the **THREE.NearestFilter**, which has a very different result:

```
colorTexture.minFilter = THREE.NearestFilter
```

If you're using a device with a pixel ratio above one, you won't see much of a difference. If not, place the camera where this face is almost hidden, and you should get more details and strange artifacts.

If you test with the **checkerboard-1024x1024.png** texture located in the **/static/textures/** folder, you will see those artefacts more clearly:

```
const colorTexture = textureLoader.load('/textures/checkerboard-1024x1024.png')
```

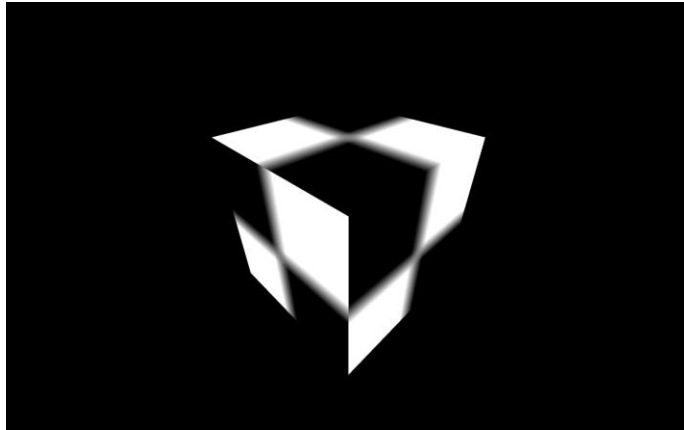
The artefacts you see are called [moiré patterns](#) and you usually want to avoid them.

### Magnification filter

The magnification filter works just like the minification filter, but when the pixels of the texture are bigger than the render's pixels. In other words, the texture too small for the surface it covers.

You can see the result using the **checkerboard-8x8.png** texture also located in the **static/textures/** folder:

```
const colorTexture = textureLoader.load('/textures/checkerboard-8x8.png')
```



The texture gets all blurry because it's a very small texture on a very large surface.

While you might think this looks awful, it is probably for the best. If the effect isn't too exaggerated, the user will probably not even notice it.

You can change the magnification filter of the texture using the **magFilter** property.

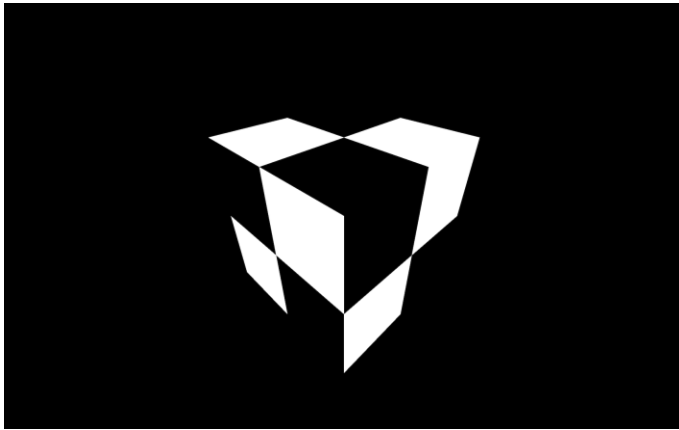
There are only two possible values:

- **THREE.NearestFilter**
- **THREE.LinearFilter**

The default is **THREE.LinearFilter**.

If you test the **THREE.NearestFilter**, you'll see that the base image is preserved, and you get a pixelated texture:

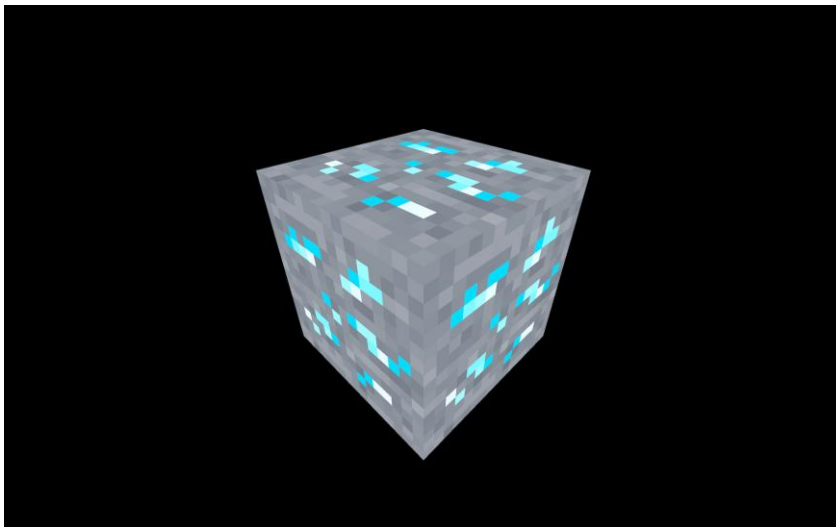
```
colorTexture.magFilter = THREE.NearestFilter
```



It can be advantageous if you're going for a Minecraft style with pixelated textures.

You can see the result using the **minecraft.png** texture located in the **static/textures/** folder:

```
const colorTexture = textureLoader.load('/textures/minecraft.png')
```



One final word about all those filters is that **THREE.NearestFilter** is cheaper than the other ones, and you should get better performances when using it.

Only use the mipmaps for the **minFilter** property. If you are using the **THREE.NearestFilter**, you don't need the mipmaps, and you can deactivate them with **colorTexture.generateMipmaps = false**:

```
colorTexture.generateMipmaps = false  
colorTexture.minFilter = THREE.NearestFilter
```

That will slightly offload the GPU.

## Texture format and optimization

When you are preparing your textures, you must keep 3 crucial elements in mind:

- The weight
- The size (or the resolution)
- The data

### The weight

Don't forget that the users going to your website will have to download those textures. You can use most of the types of images we use on the web like **.jpg** (lossy compression but usually lighter) or **.png** (lossless compression but usually heavier).

Try to apply the usual methods to get an acceptable image but as light as possible. You can use compression websites like [TinyPNG](#) (also works with jpg) or any software.

### The size

Each pixel of the textures you are using will have to be stored on the GPU regardless of the image's weight. And like your hard drive, the GPU has storage limitations. It's even worse because the automatically generated mipmapping increases the number of pixels that have to be stored.

Try to reduce the size of your images as much as possible.

If you remember what we said about the mipmapping, Three.js will produce a half smaller version of the texture repeatedly until it gets a 1x1 texture. Because of that, your texture width and height must be a power of 2. That is mandatory so that Three.js can divide the size of the texture by 2.

Some examples: **512x512**, **1024x1024** or **512x2048**

**512**, **1024** and **2048** can be divided by 2 until it reaches 1.

If you are using a texture with a width or height different than a power of 2 value, Three.js will try to stretch it to the closest power of 2 number, which can have visually poor results, and you'll also get a warning in the console.

### The data

We haven't tested it yet, because we have other things to see first, but textures support transparency. As you may know, jpg files don't have an alpha channel, so you might prefer using a png.

Or you can use an alpha map, as we will see in a future lesson.

If you are using a normal texture (the purple one), you will probably want to have the exact values for each pixel's red, green, and blue channels, or you might end up with visual glitches. For that, you'll need to use a png because its lossless compression will preserve the values.

## Where to find textures

Unfortunately, it's always hard to find the perfect textures. There are many websites, but the textures aren't always right, and you may have to pay.

It's probably a good idea to start by searching on the web. Here are some websites I frequently end up on.

- [polyigon.com](https://polyigon.com)
- [3dtextures.me](https://3dtextures.me)
- [arroway-textures.ch](https://arroway-textures.ch)

Always make sure that you have the right to use the texture if it's not for personal usage.

You can also create your own using photos and 2D software like Photoshop or even procedural textures with software like [Substance Designer](#).

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 10) MATERIALS

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

---

### Introduction

Materials are used to color each visible pixel of the geometries.

The algorithms that decide on the color of each pixel are written in programs called shaders. Writing shaders is one of the most challenging parts of WebGL and Three.js, but don't worry; Three.js has many built-in materials with pre-made shaders.

We will discover how to create our shaders in a future lesson. For now, let's use Three.js materials.

### Setup

The starter doesn't contain any object. This is an excellent occasion to revise the basics of creating [Meshes](#).

### Prepare our scene

To test the materials, we should prepare a lovely scene and load some textures.

Create 3 [Meshes](#) composed of 3 different geometries (a sphere, a plane, and a torus) and use the same [MeshBasicMaterial](#) on all of the 3. Yes, you can use one material on multiple meshes. Move the sphere on the left and the torus on the right to separate them.

The **add(...)** method support adding multiple objects at once:

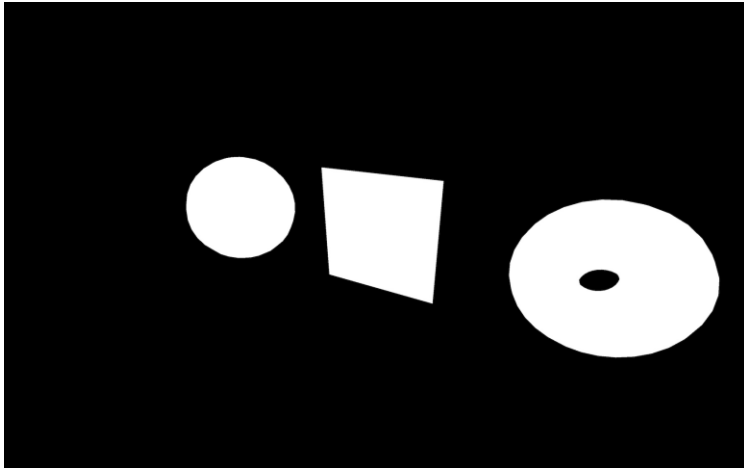
```
/**
 * Objects
 */
const material = new THREE.MeshBasicMaterial()

const sphere = new THREE.Mesh(
  new THREE.SphereGeometry(0.5, 16, 16),
  material
)
sphere.position.x = - 1.5

const plane = new THREE.Mesh(
  new THREE.PlaneGeometry(1, 1),
  material
)

const torus = new THREE.Mesh(
  new THREE.TorusGeometry(0.3, 0.2, 16, 32),
  material
)
torus.position.x = 1.5

scene.add(sphere, plane, torus)
```



We can now rotate our objects on our **tick** function as we did on the Animation lesson:

```
/**
 * Animate
 */
const clock = new THREE.Clock()

const tick = () =>
{
    const elapsedTime = clock.getElapsedTime()

    // Update objects
    sphere.rotation.y = 0.1 * elapsedTime
    plane.rotation.y = 0.1 * elapsedTime
    torus.rotation.y = 0.1 * elapsedTime

    sphere.rotation.x = 0.15 * elapsedTime
    plane.rotation.x = 0.15 * elapsedTime
    torus.rotation.x = 0.15 * elapsedTime

    // ...
}

tick()
```

You should see your 3 objects spinning slowly.

The materials we will discover use textures in many different ways. Let's load some textures using the [TextureLoader](#) as we did on the Textures lesson.

All the texture images are located in the **/static/textures/** folder. For now, we will load all the door textures located in the **/static/textures/door/** folder, the first matcap texture located in the **/static/textures/matcaps/** folder and the first gradient texture located in the **/static/textures/gradients/** folder.

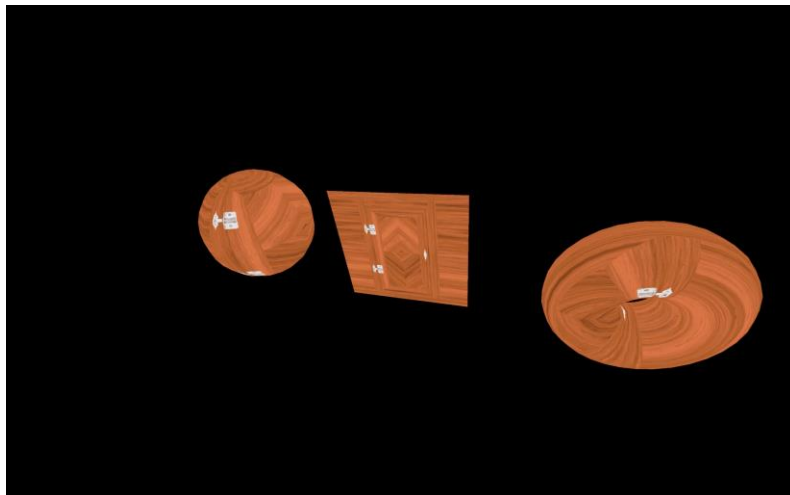
Make sure to do that before instantiating the **material**:

```
/**
 * Textures
 */
const textureLoader = new THREE.TextureLoader()

const doorColorTexture = textureLoader.load('/textures/door/color.jpg')
const doorAlphaTexture = textureLoader.load('/textures/door/alpha.jpg')
const doorAmbientOcclusionTexture =
textureLoader.load('/textures/door/ambientOcclusion.jpg')
const doorHeightTexture = textureLoader.load('/textures/door/height.jpg')
const doorNormalTexture = textureLoader.load('/textures/door/normal.jpg')
const doorMetalnessTexture =
textureLoader.load('/textures/door/metalness.jpg')
const doorRoughnessTexture =
textureLoader.load('/textures/door/roughness.jpg')
const matcapTexture = textureLoader.load('/textures/matcaps/1.png')
const gradientTexture = textureLoader.load('/textures/gradients/3.jpg')
```

To ensure that all the textures are well loaded, you can use them with the map property on your material, as we saw in the Textures lesson.

```
const material = new THREE.MeshBasicMaterial({ map: doorColorTexture })
```



Until now, we only used the [MeshBasicMaterial](#), which applies a uniform color or texture to our geometry.

If you search for "material" on the [Three.js documentation](#), you'll see that there are many classes we can use. Let's try them.

## MeshBasicMaterial

[MeshBasicMaterial](#) is probably the most "basic" material... But there are multiple properties that we haven't cover yet.

You can set most of those properties while instancing the material in the object we send as a parameter, but you can also change those properties on the instance directly:

```
const material = new THREE.MeshBasicMaterial({
  map: doorColorTexture
})

// Equals
const material = new THREE.MeshBasicMaterial()
material.map = doorColorTexture
```

We will use the second method, but feel free to do as you like.

## Map

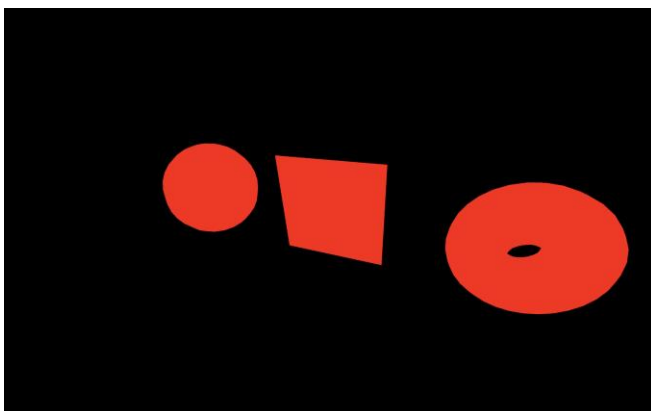
The **map** property will apply a texture on the surface of the geometry:

```
material.map = doorColorTexture
```

## Color

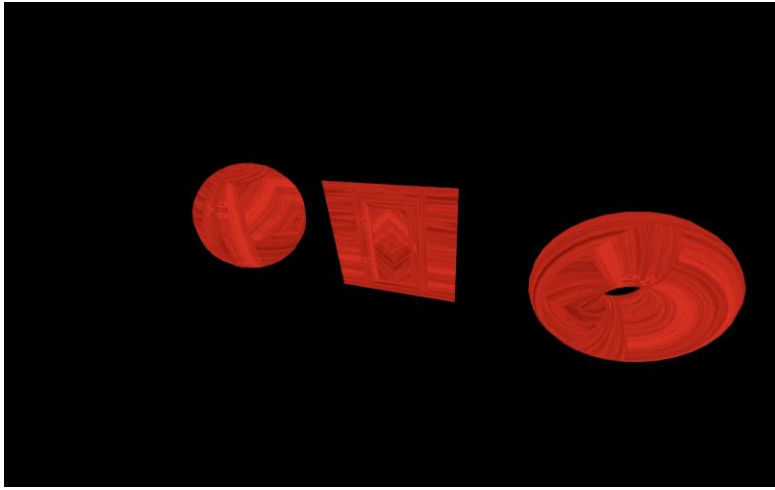
The **color** property will apply a uniform color on the surface of the geometry. When you are changing the **color** property directly, you must instantiate a [Color](#) class. You can use many different formats:

```
material.color = new THREE.Color('#ff0000')
material.color = new THREE.Color('#f00')
material.color = new THREE.Color('red')
material.color = new THREE.Color('rgb(255, 0, 0)')
material.color = new THREE.Color(0xff0000)
```



Combining **color** and **map** will tint the texture with the color:

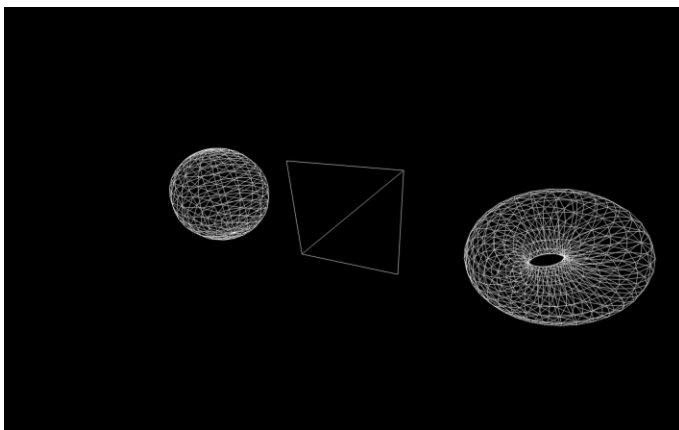
```
material.map = doorColorTexture
material.color = new THREE.Color('#ff0000')
```



## Wireframe

The **wireframe** property will show the triangles that compose your geometry with a thin line of 1px regardless of the distance of the camera:

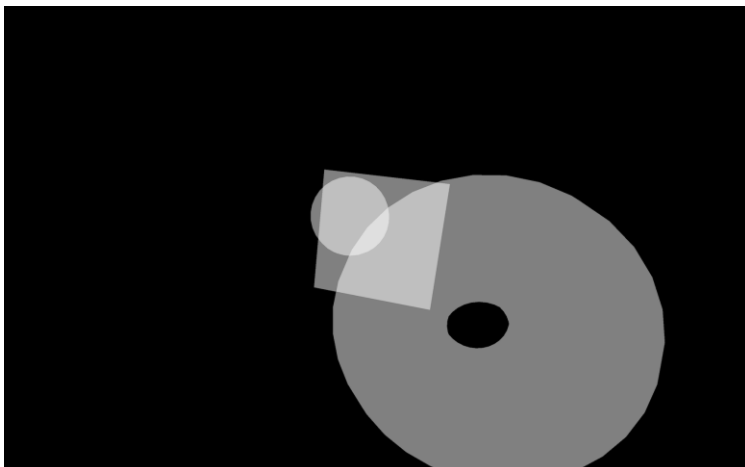
```
material.wireframe = true
```



## Opacity

The **opacity** property controls the transparency but, to work, you should set the **transparent** property to **true** to inform Three.js that this material now supports transparency:

```
material.transparent = true  
material.opacity = 0.5
```



## AlphaMap

Now that the transparency is working, we can use the **alphaMap** property to control the transparency with a texture:

```
material.transparent = true  
material.alphaMap = doorAlphaTexture
```



## Side

The **side** property lets you decide which side of a face is visible. By default, the front side is visible (**THREE.FrontSide**), but you can show the backside instead (**THREE.BackSide**) or both (**THREE.DoubleSide**):

```
material.side = THREE.DoubleSide
```

You should see both the front and the back of the plane.

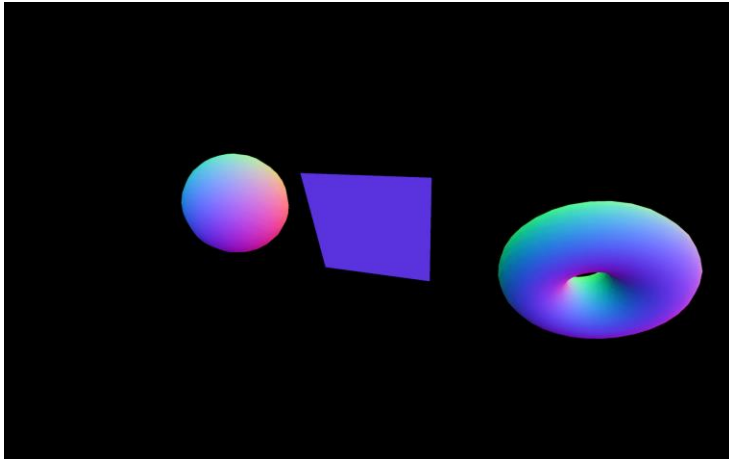
Try to avoid using **THREE.DoubleSide** because rendering both sides means having twice more triangles to render.

Some of these properties like **wireframe** or **opacity** can be used with other types of materials. We won't repeat those every time.

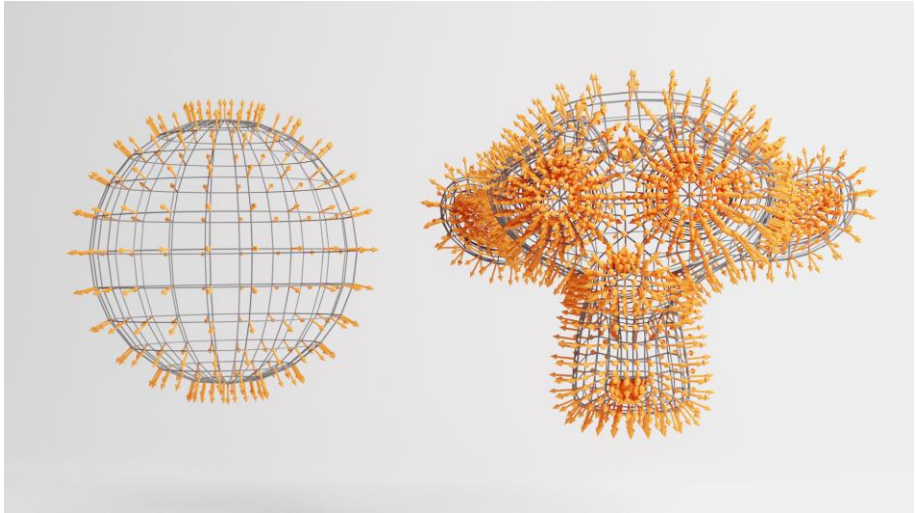
## MeshNormalMaterial

The [MeshNormalMaterial](#) displays a nice purple, blueish, greenish color that looks like the normal texture we saw in the Textures lessons. That is no coincidence because both are related to what we call normals:

```
const material = new THREE.MeshNormalMaterial()
```



Normals are information encoded in each vertex that contains the direction of the outside of the face. If you displayed those normals as arrows, you would get straight lines coming out of each vertex that composes your geometry.

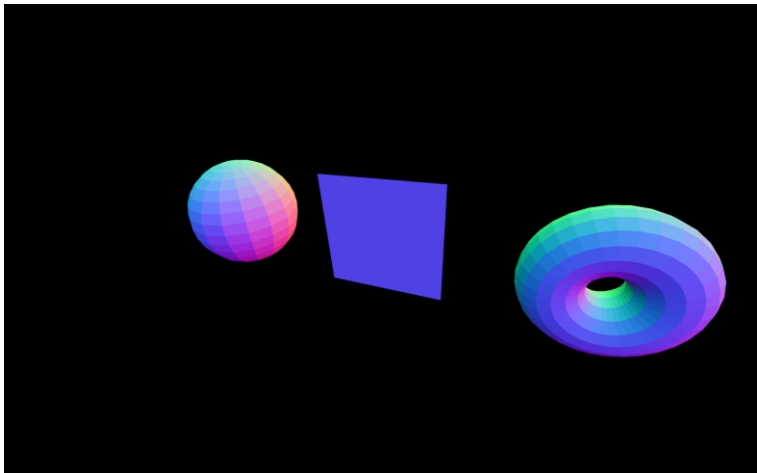


You can use Normals for many things like calculating how to illuminate the face or how the environment should reflect or refract on the geometries' surface.

When using the [MeshNormalMaterial](#), the color will just display the normal relative's orientation to the camera. If you rotate around the sphere, you'll see that the color is always the same, regardless of which part of the sphere you're looking at.

While you can use some of the properties we discovered with the [MeshBasicMaterial](#) like **wireframe**, **transparent**, **opacity** and **side**, there is also a new property that you can use, which is called **flatShading**:

```
material.flatShading = true
```



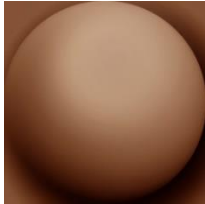
**flatShading** will flatten the faces, meaning that the normals won't be interpolated between the vertices.

[MeshNormalMaterial](#) can be useful to debug the normals, but it also looks great, and you can use it as it is just like ilithya did on her portfolio <https://www.ilithya.rocks>.

## MeshMatcapMaterial

[MeshMatcapMaterial](#) is a fantastic material because of how great it can look while being very performant.

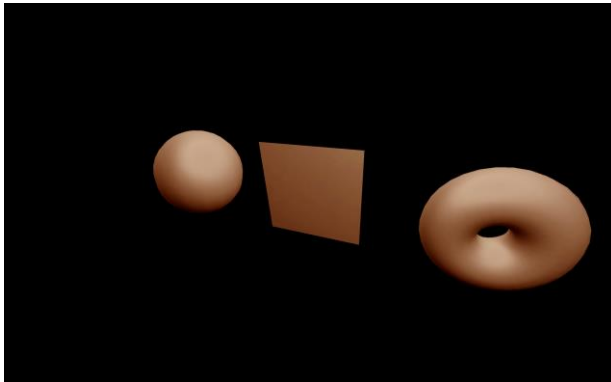
For it to work, the [MeshMatcapMaterial](#) needs a reference texture that looks like a sphere.



The material will then pick colors on the texture according to the normal orientation relative to the camera.

To set that reference matcap texture, use the **matcap** property:

```
const material = new THREE.MeshMatcapMaterial()  
material.matcap = matcapTexture
```

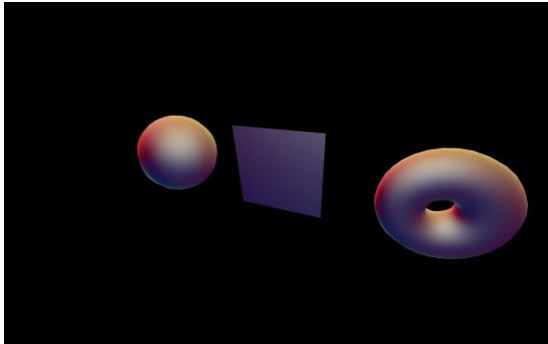


The meshes will appear illuminated, but it's just a texture that looks like it.

The only problem is that the illusion is the same regardless of the camera orientation. Also, you cannot update the lights because there are none.

Try different textures available on the **/static/textures/matcaps/** folder (just one of the lines below):

```
const matcapTexture = textureLoader.load('/textures/matcaps/2.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/3.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/4.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/5.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/6.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/7.png')  
const matcapTexture = textureLoader.load('/textures/matcaps/8.png')
```



Regarding where to find matcaps textures, you can do a simple search on the web like any type of textures. Make sure you have the right to use the texture if it's not for personal usage. There is also this vast list of matcaps: <https://github.com/nidorx/matcaps>

You can also create your own matcaps using a 3D software by rendering a sphere in front of the camera in a square image. Finally, you can try to make a matcap in 2D software like Photoshop.

## MeshDepthMaterial

The [MeshDepthMaterial](#) will simply color the geometry in white if it's close to the camera's **near** value and in black if it's close to the **far** value of the camera:

```
const material = new THREE.MeshDepthMaterial()
```

You can use this material for special effects where you need to know how far the pixel is from the camera. We will use it in a future lesson.

## Adding a few lights

The following materials need lights to be seen. Let's add two simple lights to our scene.

Create an [AmbientLight](#) and add it to the scene:

```
/**
 * Lights
 */
const ambientLight = new THREE.AmbientLight(0xffffff, 0.5)
scene.add(ambientLight)
```

Create a [PointLight](#) and add it to the scene:

```
// ...

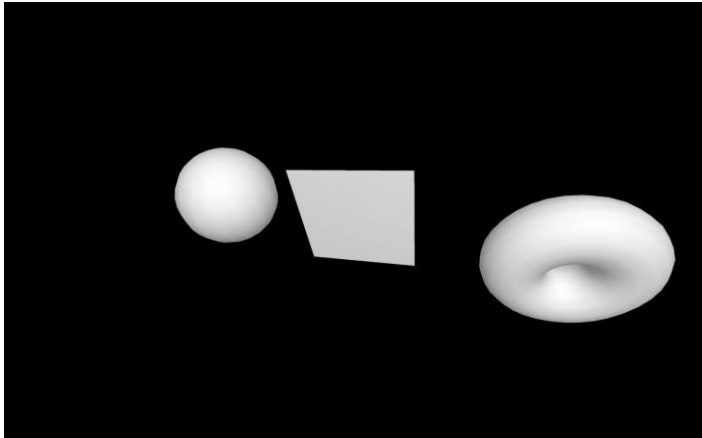
const pointLight = new THREE.PointLight(0xffffff, 0.5)
pointLight.position.x = 2
pointLight.position.y = 3
pointLight.position.z = 4
scene.add(pointLight)
```

We will see more about lights, how they work and how to tweak them in a future lesson.

## MeshLambertMaterial

The [MeshLambertMaterial](#) is the first material reacting to light that we are going to use:

```
const material = new THREE.MeshLambertMaterial()
```



Things are getting realistic, as you can see. While the illumination isn't very convincing, it's a good start.

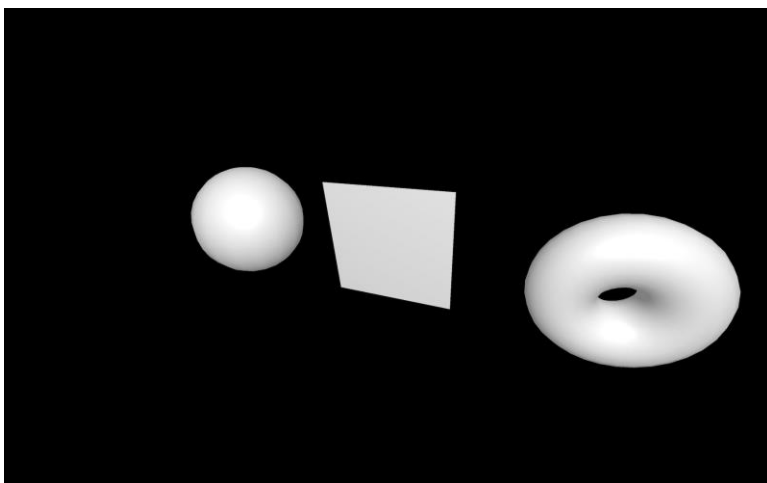
[MeshLambertMaterial](#) supports the same properties as the [MeshBasicMaterial](#) but also some properties related to lights. We will see those properties later in the lesson with more adequate materials.

The [MeshLambertMaterial](#) is the most performant material that uses lights. Unfortunately, the parameters aren't convenient, and you can see strange patterns on the geometry if you look closely at rounded geometries like the sphere.

## MeshPhongMaterial

The [MeshPhongMaterial](#) is very similar to the [MeshLambertMaterial](#), but the strange patterns are less visible, and you can also see the light reflection on the surface of the geometry:

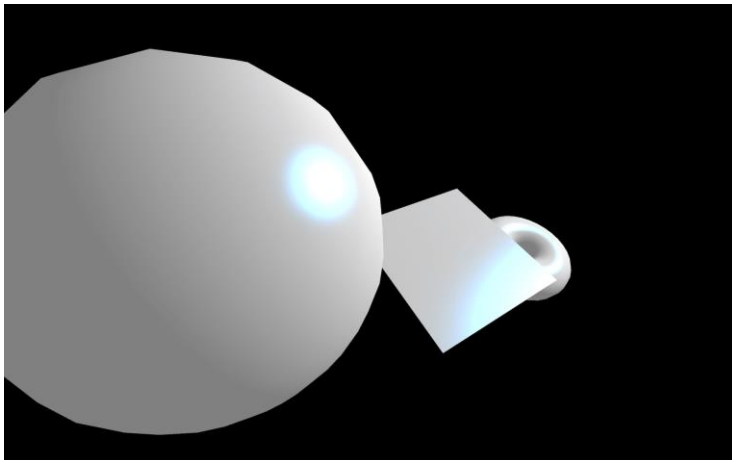
```
const material = new THREE.MeshPhongMaterial()
```



[MeshPhongMaterial](#) is less performant than [MeshLambertMaterial](#). However, it doesn't really matter at this level.

You can control the light reflection with the **shininess** property. The higher the value, the shinier the surface. You can also change the color of the reflection by using the **specular** property:

```
material.shininess = 100  
material.specular = new THREE.Color(0x1188ff)
```

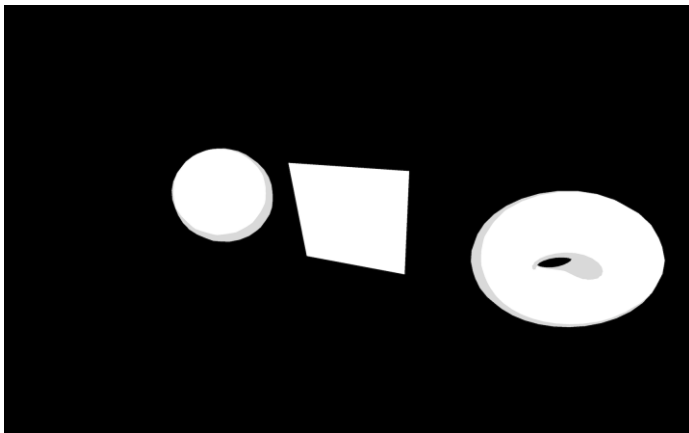


The light reflection will have a blue-ish color.

## MeshToonMaterial

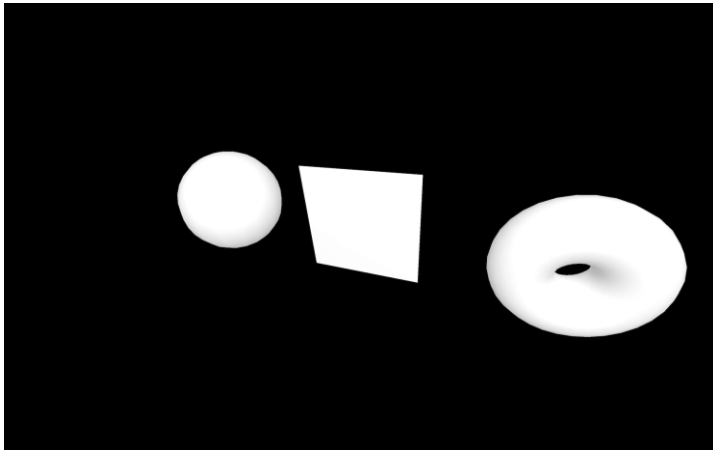
The [MeshToonMaterial](#) is similar to the [MeshLambertMaterial](#) in terms of properties but with a cartoonish style:

```
const material = new THREE.MeshToonMaterial()
```



By default, you only get a two parts coloration (one for the shadow and one for the light). To add more steps to the coloration, you can use the **gradientMap** property and use the **gradientTexture** we loaded at the start of the lesson:

```
material.gradientMap = gradientTexture
```

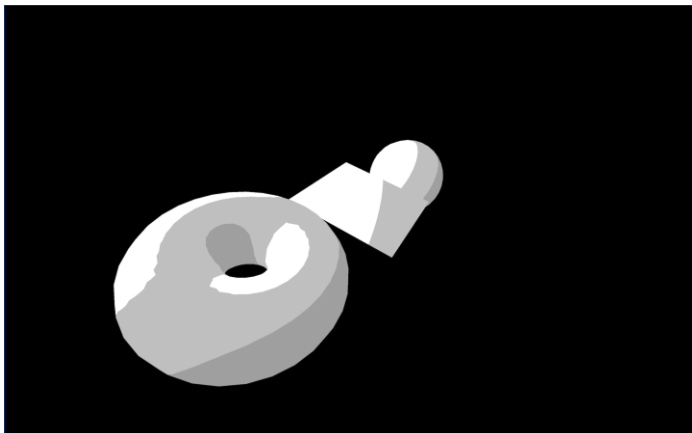


If you test this, you'll see that the cartoon effect doesn't work anymore. That is because the gradient texture we used is tiny, and the pixels of that texture are blended. Yes, it's a matter of **minFilter**, **magFilter**, and **mipmapping** like we saw in the Textures lesson.

To fix this, we can simply change the **minFilter** and **magFilter** to **THREE.NearestFilter**.

Using **THREE.NearestFilter** means that we are not using the mip mapping, we can deactivate it with **gradientTexture.generateMipmaps = false**:

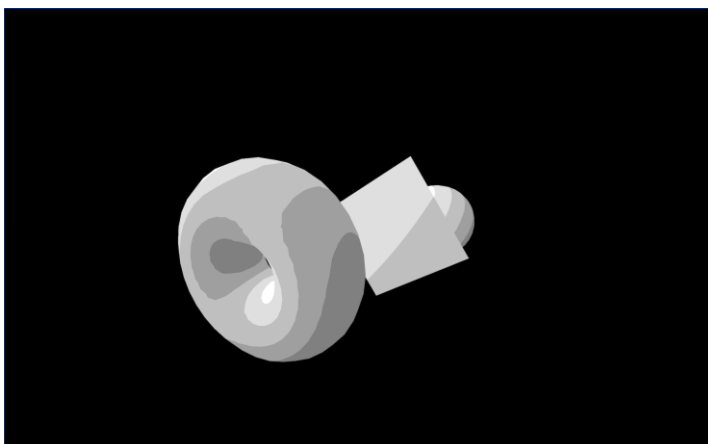
```
gradientTexture.minFilter = THREE.NearestFilter
gradientTexture.magFilter = THREE.NearestFilter
gradientTexture.generateMipmaps = false
```



You should now see the cartoon effect with an intermediate step.

You can try with even more steps by using the image located in **/static/textures/gradients.5.jpg**:

```
const gradientTexture = textureLoader.load('/textures/gradients/5.jpg')
```

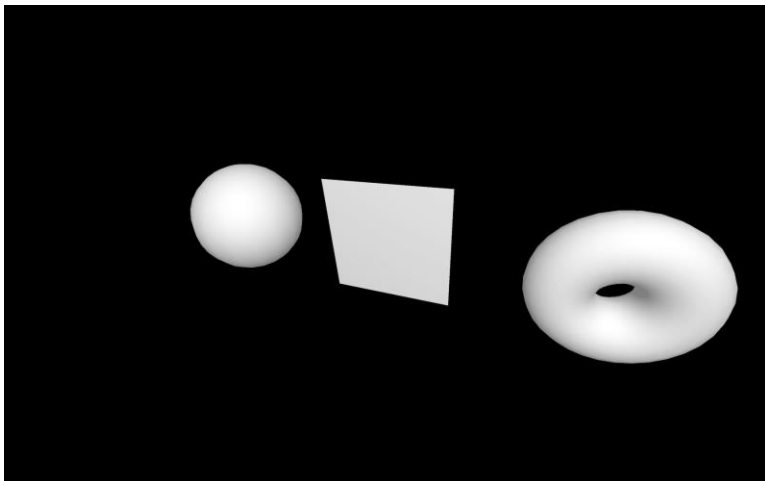


## MeshStandardMaterial

The [MeshStandardMaterial](#) uses physically based rendering principles. Yes, we are talking about the PBR we saw in the Textures lesson. Like the [MeshLambertMaterial](#) and the [MeshPhongMaterial](#), it supports lights but with a more realistic algorithm and better parameters like roughness and metalness.

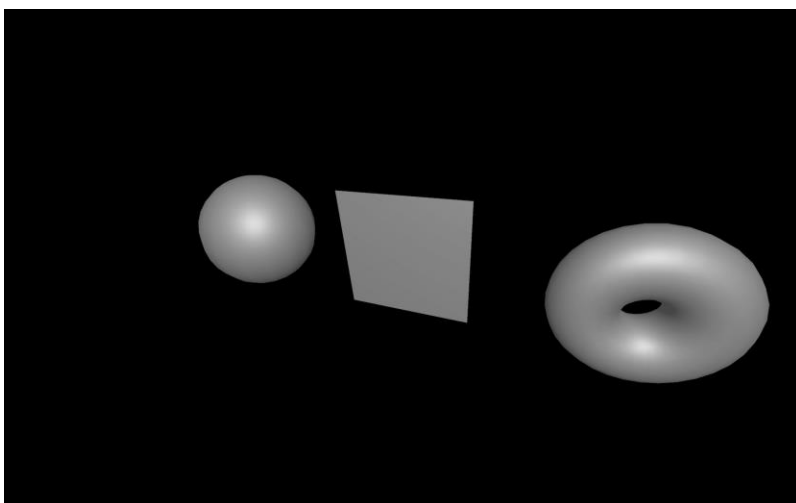
It's called "standard" because the PBR is becoming a standard in many software, engines, and libraries. The idea is to have a realistic result with realistic parameters, and you should have a very similar result regardless of the technology you are using:

```
const material = new THREE.MeshStandardMaterial()
```



You can change the **roughness** and the **metalness** properties directly:

```
material.metalness = 0.45  
material.roughness = 0.65
```



## Add a debug UI

While this isn't required, now would be an excellent time to add a debug UI. That will be very useful to test the different properties.

First, we must add the [Dat.GUI](#) dependency to our project. In the terminal, on the project folder (where the server should be currently running), use the following command:

```
npm install --save lil-gui
```

As mentioned in the **Debug** lesson, we are installing **lil-gui** instead of `dat.gui`. This library works exactly the same as `dat.gui` and we will refer to it as "dat.gui" in the rest of the course.

Then, on top of your code, import **lil-gui** (don't forget to re-launch the server with **npm run dev** if you stopped it):

```
import * as dat from 'lil-gui'
```

You can now create an instance of it:

```
/**
 * Debug
 */
const gui = new dat.GUI()
```

And add the tweaks (after creating the material):

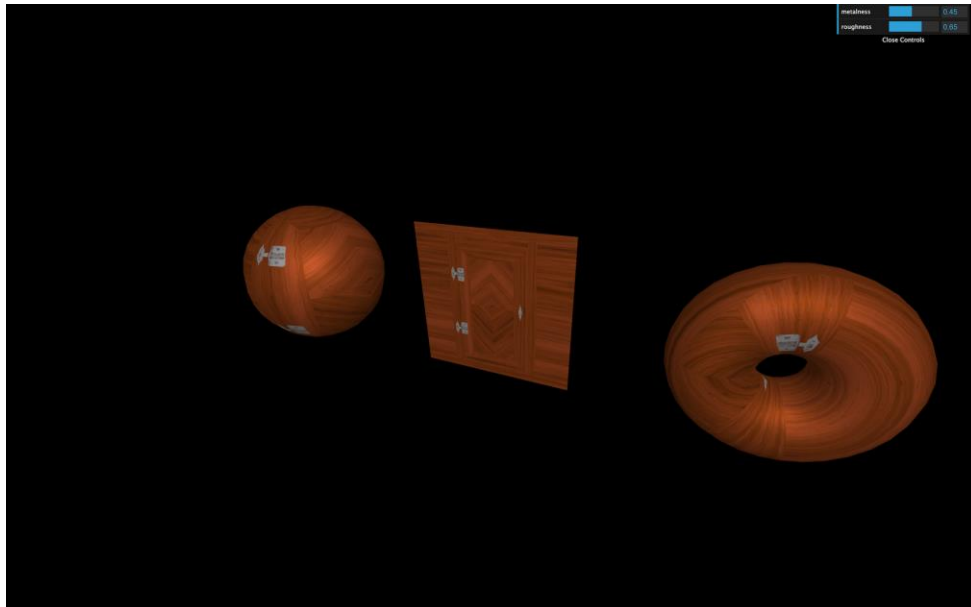
```
gui.add(material, 'metalness').min(0).max(1).step(0.0001)
gui.add(material, 'roughness').min(0).max(1).step(0.0001)
```

And that's it. You can now change the **metalness** and the **roughness** as you like.

Let's continue with the other properties of the [MeshStandardMaterial](#).

The **map** property allows you to apply a simple texture. You can use **doorColorTexture**:

```
material.map = doorColorTexture
```



The **aoMap** property (literally "ambient occlusion map") will add shadows where the texture is dark. For it to work, you must add what we call a second set of UV (the coordinates that help position the textures on the geometries).

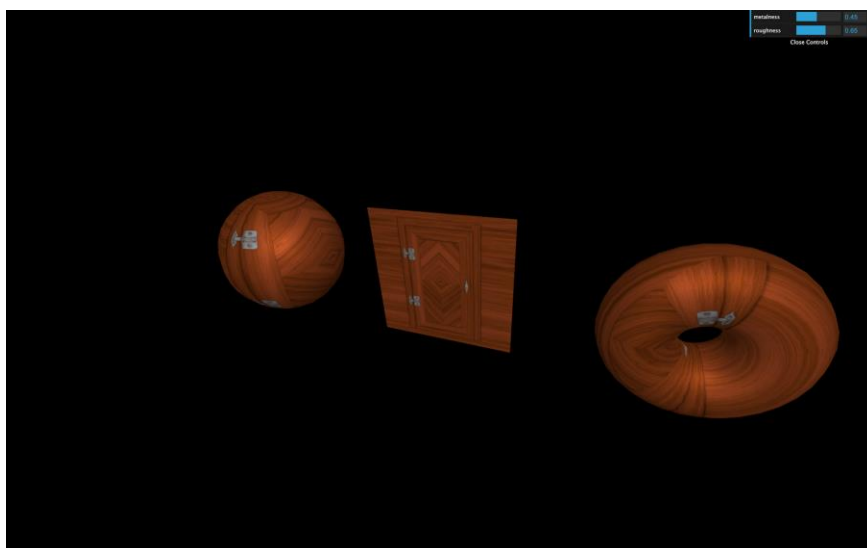
We can simply add new attributes like we did on the Geometries lesson and use the default **uv** attribute. In more simple terms, we duplicated the **uv** attribute.

Call this new attribute **uv2**:

```
sphere.geometry.setAttribute('uv2', new
THREE.BufferAttribute(sphere.geometry.attributes.uv.array, 2))
plane.geometry.setAttribute('uv2', new
THREE.BufferAttribute(plane.geometry.attributes.uv.array, 2))
torus.geometry.setAttribute('uv2', new
THREE.BufferAttribute(torus.geometry.attributes.uv.array, 2))
```

You can now add the **aoMap** using the **doorAmbientOcclusionTexture** texture and control the intensity using the **aoMapIntensity** property:

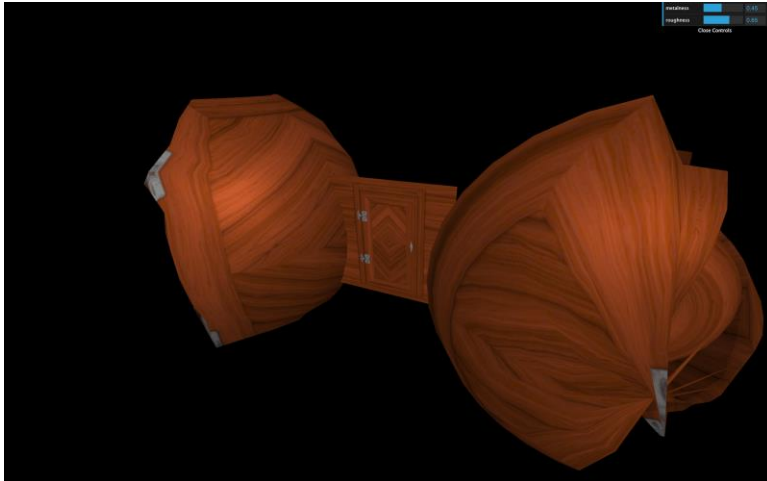
```
material.aoMap = doorAmbientOcclusionTexture
material.aoMapIntensity = 1
```



The crevices should look darker, which creates contrast and adds dimension.

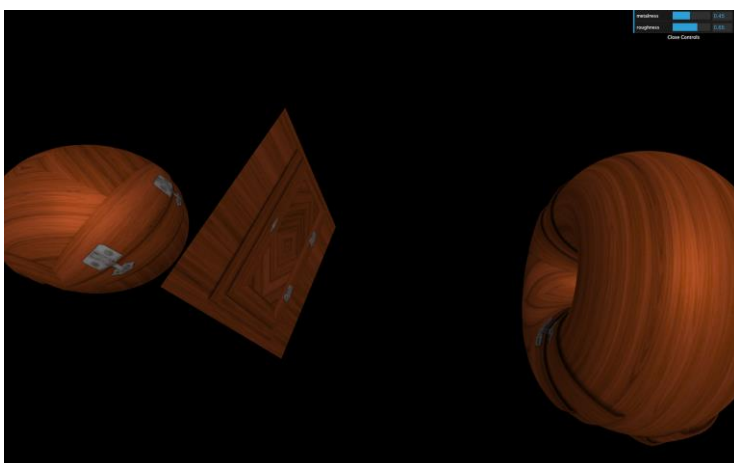
The **displacementMap** property will move the vertices to create true relief:

```
material.displacementMap = doorHeightTexture
```



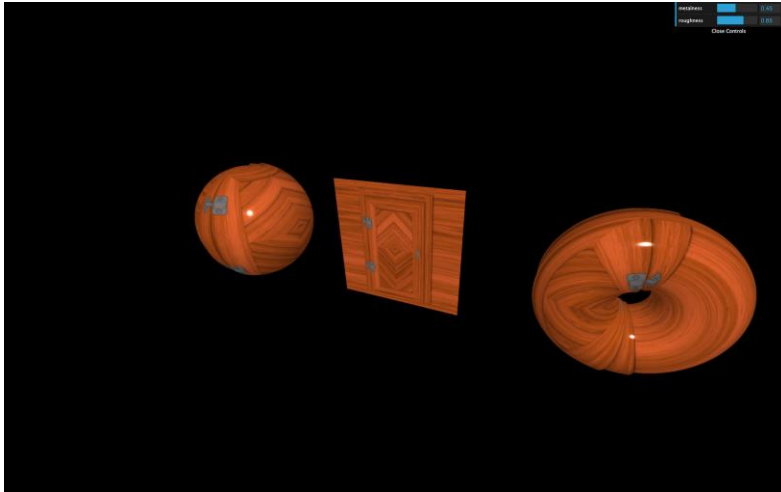
It should look terrible. That is due to the lack of vertices on our geometries (we need more subdivisions) and the displacement being way too strong:

```
material.displacementScale = 0.05  
  
// ...  
new THREE.SphereGeometry(0.5, 64, 64),  
  
// ...  
new THREE.PlaneGeometry(1, 1, 100, 100),  
  
// ...  
new THREE.TorusGeometry(0.3, 0.2, 64, 128),
```



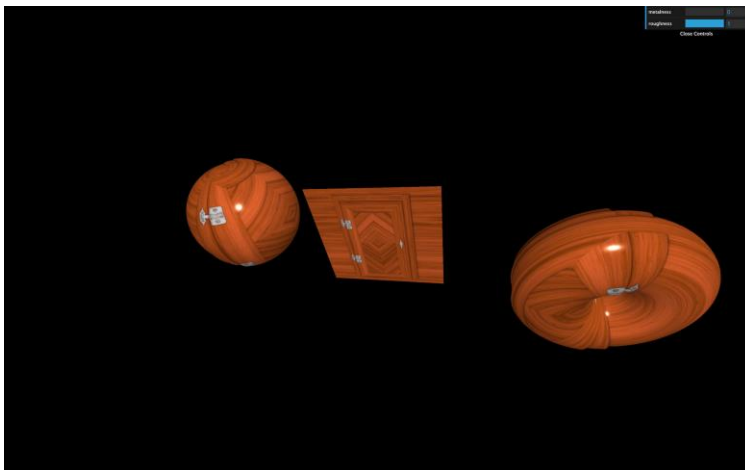
Instead of specifying uniform **metalness** and **roughness** for the whole geometry, we can use **metalnessMap** and **roughnessMap**:

```
material.metalnessMap = doorMetalnessTexture  
material.roughnessMap = doorRoughnessTexture
```



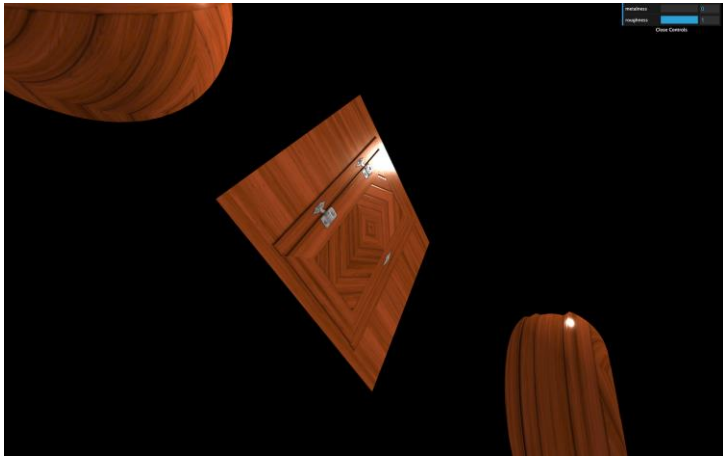
The reflection looks weird because the **metalness** and **roughness** properties still affect each map respectively. We should comment them or use their original values:

```
material.metalness = 0  
material.roughness = 1
```



The **normalMap** will fake the normal orientation and add details on the surface regardless of the subdivision:

```
material.normalMap = doorNormalTexture
```

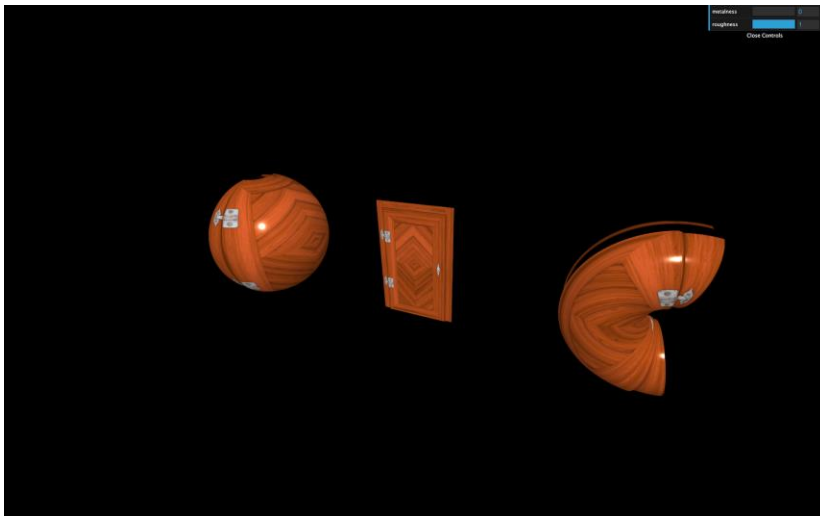


You can change the normal intensity with the **normalScale** property. Be careful, it's a [Vector2](#):

```
material.normalScale.set(0.5, 0.5)
```

And finally, you can control the alpha using the **alphaMap** property. Don't forget to set the **transparent** property to **true**:

```
material.transparent = true  
material.alphaMap = doorAlphaTexture
```



Here's a beautiful door. Feel free to tweak the properties and try stuff.

## MeshPhysicalMaterial

The [MeshPhysicalMaterial](#) is the same as the [MeshStandardMaterial](#) but with support of a clear coat effect. You can control that clear coat's properties and even use a texture as in this [Three.js example](#), but we won't try this one here.

## PointsMaterial

You can use [PointsMaterial](#) with particles. We will see more about that in a dedicated lesson.

## ShaderMaterial and RawShaderMaterial

[ShaderMaterial](#) and [RawShaderMaterial](#) can both be used to create your materials but we will see more about that in a dedicated lesson.

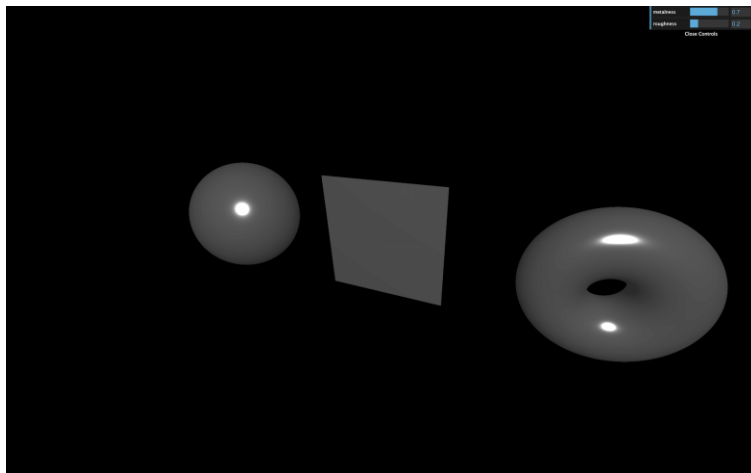
## Environment map

The environment map is like an image of what's surrounding the scene. You can use it to add reflection or refraction to your objects. It can also be used as lighting information.

We haven't covered it yet, but you can use it with many of the materials we saw.

First, let's setup a very simple [MeshStandardMaterial](#) with the debug UI as we did earlier:

```
const material = new THREE.MeshStandardMaterial()
material.metalness = 0.7
material.roughness = 0.2
gui.add(material, 'metalness').min(0).max(1).step(0.0001)
gui.add(material, 'roughness').min(0).max(1).step(0.0001)
```



To add the environment map to our material, we must use the **envMap** property. Three.js only supports cube environment maps. Cube environment maps are 6 images with each one corresponding to a side of the environment.

You can find multiple environment maps in the `/static/textures/environmentMap/` folder.

To load a cube texture, you must use the [CubeTextureLoader](#) instead of the [TextureLoader](#).

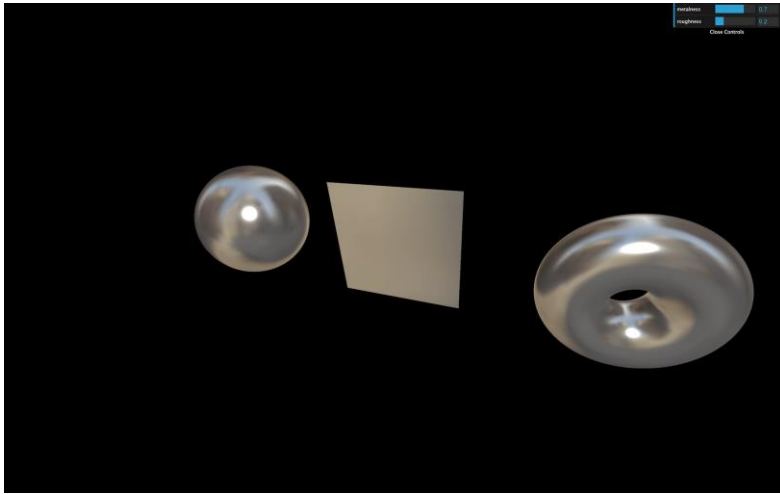
Instantiate the [CubeTextureLoader](#) before instantiating the **material** and call its **load(...)** method but use an array of paths instead of one path:

```
const cubeTextureLoader = new THREE.CubeTextureLoader()

const environmentMapTexture = cubeTextureLoader.load([
  '/textures/environmentMaps/0/px.jpg',
  '/textures/environmentMaps/0/nx.jpg',
  '/textures/environmentMaps/0/py.jpg',
  '/textures/environmentMaps/0/ny.jpg',
  '/textures/environmentMaps/0/pz.jpg',
  '/textures/environmentMaps/0/nz.jpg'
])
```

You can now use the **environmentMapTexture** in the **envMap** property of your material:

```
material.envMap = environmentMapTexture
```



You should see the environment reflect on the surface of the geometry. Try to tweak the **metalness** and **roughness** for different results.

You can also test the other environment maps in the `/static/textures/environmentMap/` folder.

### Where to find environment maps

To find cool environment maps, you can always do a simple search on the web and make sure you have the right to use the environment map if it's not for personal usage.

One of the best sources is [HDRIHaven](https://hdrihaven.com/). This website has hundreds of awesome HDRIs. HDRI stands for **High Dynamic Range Imaging**. They are composed of one image (not a cube map) and contain more data than a simple image, thus improving lighting information for a more realistic result. [HDRIHaven](https://hdrihaven.com/) images are free and under [CC0 license](https://creativecommons.org/licenses/by/4.0/), which means that you can do anything you want with them without having to credit the authors. But if you appreciate their work, you can thank them by subscribing to [their Patreon](https://www.patreon.com/hdrihaven).

But we have a problem. As we said, Three.js only supports cube maps. To convert an HDRI to a cube map, you can use this online tool: <https://matheowis.github.io/HDRI-to-CubeMap/>

Upload an HDRI, rotate it as you please, and download a cubemap version composed of 6 images. The default format is **.png**, and you'll have to convert them to **.jpg** if you want.

## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 11) 3D TEXT

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

---

### Introduction

We already know enough basics to create some content. For our first project, we will re-create what ilithya did with her cool portfolio <https://www.ilithya.rocks/> and have a big 3D text in the middle of the scene with objects floating around.

This portfolio is an excellent example of what you can do quite early when learning Three.js. It simple, efficient, and it looks great.

Three.js already supports 3D text geometries with the [TextGeometry](#) class. The problem is that you must specify a font, and this font must be in a particular json format called typeface.

We won't talk about licenses, but you must have the right to use the font unless it's for personal usage.

### How to get a typeface font

There are many ways of getting fonts in that format. First, you can convert your font with converters like this one: <https://gero3.github.io/facetype.js/>. You have to provide a file and click on the convert button.

You can also find fonts in the Three.js examples located in the `/node_modules/three/examples/fonts/` folder. You can take those fonts and put them in the `/static/` folder, or you can import them directly in your JavaScript file because they are json and .json files are supported just like .js files in Vite:

```
import typefaceFont from  
'three/examples/fonts/helvetiker_regular.typeface.json'
```

We will mix those two techniques by opening the `/node_modules/three/examples/fonts/`, taking the `helvetiker_regular.typeface.json` and `LICENSE` files, and putting these in the `/static/fonts/` folder (that you need to create).

The font is now accessible just by writing `/fonts/helvetiker_regular.typeface.json` at the end of the base URL.

### Load the font

To load the font, we must use a new loader class called [FontLoader](#).

This class is not available in the **THREE** variable. We need to import it like we did with the **OrbitControls** class earlier in the course:

```
import { FontLoader } from 'three/examples/jsm/loaders/FontLoader.js'
```

This loader works like the [TextureLoader](#). Add the following code after the **textureLoader** part (if you are using another font, don't forget to change the path):

```
/**
 * Fonts
 */
const fontLoader = new FontLoader()

fontLoader.load(
  '/fonts/helvetiker_regular.typeface.json',
  (font) =>
  {
    console.log('loaded')
  }
)
```

You should get **'loaded'** in you console. If not, check the previous steps and search for potential errors in the console.

We now have access to the font by using the **font** variable inside the function. Unlike the [TextureLoader](#), we have to write the rest of our code inside that success function.

## Create the geometry

As we said earlier, we are going to use [TextGeometry](#).

Like for the [FontLoader](#), we need to import it:

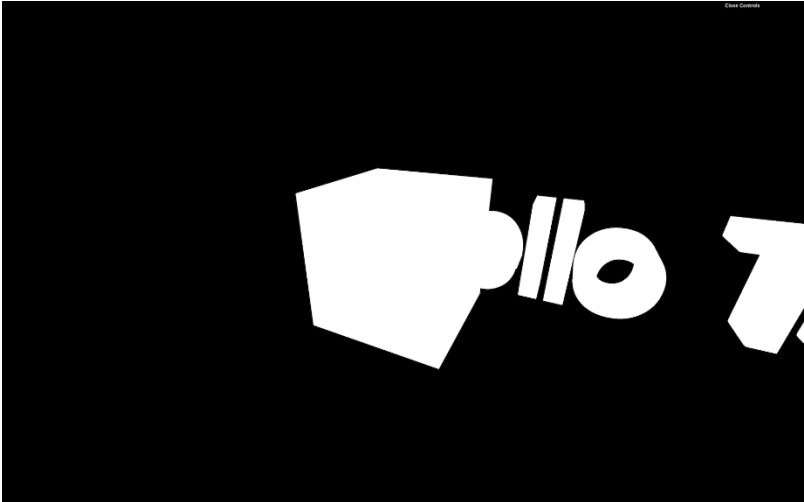
```
import { TextGeometry } from 'three/examples/jsm/geometries/TextGeometry.js'
```

Be careful with the example code on the [documentation page](#); the values are much bigger than those in our scene.

Make sure to write your code inside the success function:

```
fontLoader.load(
  '/fonts/helvetiker_regular.typeface.json',
  (font) =>
  {
    const textGeometry = new TextGeometry(
      'Hello Three.js',
      {
        font: font,
        size: 0.5,
        height: 0.2,
        curveSegments: 12,
        bevelEnabled: true,
        bevelThickness: 0.03,
        bevelSize: 0.02,
        bevelOffset: 0,
        bevelSegments: 5
      }
    )
    const textMaterial = new THREE.MeshBasicMaterial()
    const text = new THREE.Mesh(textGeometry, textMaterial)
    scene.add(text)
```

```
)  
}
```



You should get a white 3D text that needs improvements.

First, get rid of the cube. Its purpose was to make sure that everything is working.



If you want to see something cool, add **wireframe: true** to your material.

```
const textMaterial = new THREE.MeshBasicMaterial({ wireframe: true })
```



You can now see how the geometry is generated, and there are a lot of triangles. Creating a text geometry is long and hard for the computer. Avoid doing it too many times and keep the geometry as low poly as possible by reducing the **curveSegments** and **bevelSegments** properties.

Remove the **wireframe** once you're happy with the level of details.

## Center the text

There are several ways to center the text. One way of doing it is by using bounding. The bounding is the information associated with the geometry that tells what space is taken by that geometry. It can be a box or a sphere.



You cannot actually see those bindings, but it helps Three.js easily calculate if the object is on the screen, and if not, the object won't even be rendered. That is called frustum culling, but it's not the subject of this lesson.

What we want is to use this bounding to know the size of the geometry and recenter it. By default, Three.js is using sphere bounding. What we want is a box bounding, to be more precise. To do so, we can ask Three.js to calculate this box bounding by calling **computeBoundingBox()** on the geometry:

```
textGeometry.computeBoundingBox()
```

And we can check this box with the **boundingBox** property on the geometry.

```
console.log(textGeometry.boundingBox)
```

The result is an object called [Box3](#) that has a **min** property and a **max** property. The **min** property isn't at **0** as we could have expected. That is due to the **bevelThickness** and **bevelSize**, but we can ignore this for now.

Now that we have the measures, we can move the object. Instead of moving the mesh, we are going to move the whole geometry. This way, the mesh will still be in the center of the scene, but the text geometry will also be centered inside our mesh.

To do this, we can use the **translate(...)** method on our geometry right after the **computeBoundingBox()** method:

```
textGeometry.translate(  
  - textGeometry.boundingBox.max.x * 0.5,  
  - textGeometry.boundingBox.max.y * 0.5,  
  - textGeometry.boundingBox.max.z * 0.5
```

```
)
```



The text should be centered but if you want to be very precise, you should also subtract the **bevelSize** which is **0.02**:

```
textGeometry.translate(  
  - (textGeometry.boundingBox.max.x - 0.02) * 0.5, // Subtract bevel size  
  - (textGeometry.boundingBox.max.y - 0.02) * 0.5, // Subtract bevel size  
  - (textGeometry.boundingBox.max.z - 0.03) * 0.5 // Subtract bevel  
  thickness  
)
```

What we did here can actually be done much faster by calling the **center()** method on the geometry:

```
textGeometry.center()
```

So much easier, isn't it? The point of doing it ourselves was to learn about boundings and frustum culling.

### Add a matcap material

It's time to add a cool material to our text. We are going to use a [MeshMatcapMaterial](#) because it looks cool, and it has great performance.

First, let's choose a matcap texture. We are going to use the matcaps located in the **/static/textures/matcaps/** folder but feel free to use your own matcaps.

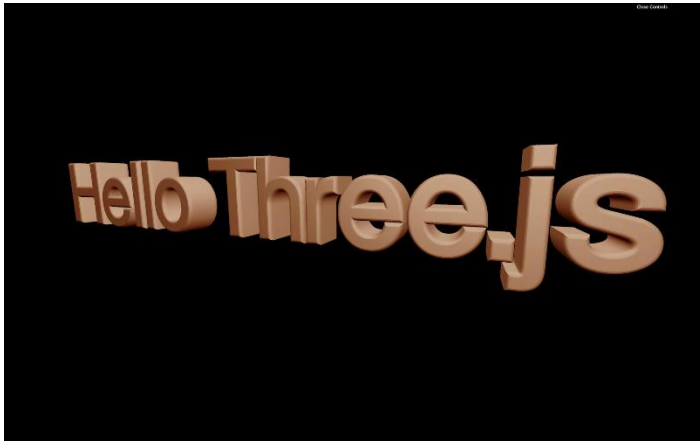
You can also download one from this repository <https://github.com/nidorx/matcaps>. Don't spend too much time choosing it! If it's not for personal usage, make sure you have the right to use it. You don't need a high-resolution texture and **256x256** should be more than enough.

We can now load the texture by using the [TextureLoader](#) already in the code:

```
const matcapTexture = textureLoader.load('/textures/matcaps/1.png')
```

We can now replace our ugly [MeshBasicMaterial](#) by a beautiful [MeshMatcapMaterial](#) and use our **matcapTexture** variable with the **matcap** property:

```
const textMaterial = new THREE.MeshMatcapMaterial({ matcap: matcapTexture })
```



You should have a lovely text with a cool looking material on it.

## Add objects

Let's add objects floating around. To do that, we will create one donut but inside a loop function.

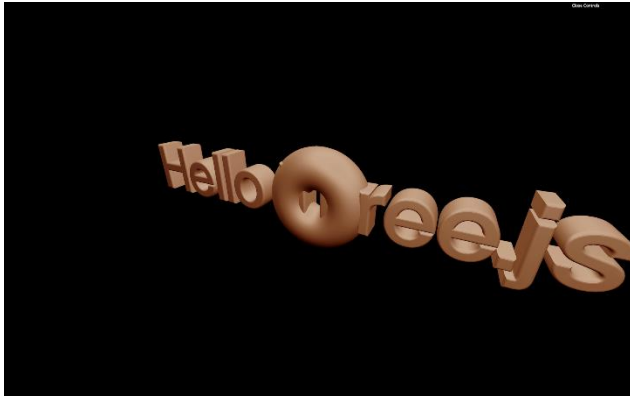
In the success function, right after the **text** part, add the loop function:

```
for(let i = 0; i < 100; i++)  
{  
  
}
```

We could have done this outside of the success function but we are going to need the text and the objects being created together for a good reason that you'll see a little later.

In this loop, create a [TorusGeometry](#) (such a technical name for a donut), the same material as for the text and the [Mesh](#):

```
for(let i = 0; i < 100; i++)  
{  
    const donutGeometry = new THREE.TorusGeometry(0.3, 0.2, 20, 45)  
    const donutMaterial = new THREE.MeshMatcapMaterial({ matcap:  
matcapTexture })  
    const donut = new THREE.Mesh(donutGeometry, donutMaterial)  
    scene.add(donut)  
}
```



You should get 100 donuts all in the same place.

Let's add some randomness for their position:

```
donut.position.x = (Math.random() - 0.5) * 10  
donut.position.y = (Math.random() - 0.5) * 10  
donut.position.z = (Math.random() - 0.5) * 10
```



You should get the 100 donuts dispersed on the scene.

Add randomness to the rotation. No need to rotate all 3 axes, and because the donut is symmetric, half of a revolution is enough:

```
donut.rotation.x = Math.random() * Math.PI  
donut.rotation.y = Math.random() * Math.PI
```



The donuts should have rotate in all directions.

Finally, we can add randomness to the scale. Be careful, though; we need to use the same value for all 3 axes (**x**, **y**, **z**):

```
const scale = Math.random()
donut.scale.set(scale, scale, scale)
```



## Optimize

Our code isn't very optimized. As we saw in a previous lesson, we can use the same material on multiple [Meshes](#), but we can also use the same geometry.

Move the **donutGeometry** and the **donutMaterial** out of the loop:

```
const donutGeometry = new THREE.TorusGeometry(0.3, 0.2, 20, 45)
const donutMaterial = new THREE.MeshMatcapMaterial({ matcap: matcapTexture })

for(let i = 0; i < 100; i++)
{
    // ...
}
```

You should get the same result, but we can go even further. The material of the **text** is the same as for the **donut**.

Let's remove the **donutMaterial**, rename the **textMaterial** by **material** and use it for both the **text** and the **donut**:

```
const material = new THREE.MeshMatcapMaterial({ matcap: matcapTexture })

// ...

const text = new THREE.Mesh(textGeometry, material)

// ...

for(let i = 0; i < 100; i++)
{
    const donut = new THREE.Mesh(donutGeometry, material)

    // ...
}
```

We could go even further, but there is a dedicated lesson about optimizations.

## Go further

If you want, you can add more shapes, animate them and even try other matcaps.



## ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 12) GO LIVE

**COURSE/SUBJECT:** APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

**COURSE NO.:** ITE-18

**INSTRUCTOR/PROF:** JAMES EARL D. CUBILLAS, M.Sc., ECT

---

### Introduction

At this point in your journey, you might want to share what you've built. A screenshot or a video is a good start, but your friends and family will enjoy an interactive version much more. It's time to make those WebGL experiences live.

### Traditional build

In the next section, we are going to use a solution that doesn't require you to subscribe to a "traditional" hosting solution like OVH, 1and1 or Gandhi where you have to upload files manually by using an FTP client. Some of you, however, might already have a host like one of these and you just want to get those files that you are supposed to upload.

You can't simply put the whole project with the **node\_modules/** folder and the Vite configuration on the host. First, you need to "build" your project within that webpack configuration in order to create HTML, CSS, JS and assets files that can be interpreted by browsers.

To build your project, run **npm run build** in the terminal.

This command will run the script located in the **/package.json** file in the **scripts > build** property.

Wait a few seconds and the files should be available in the **/dist/** folder which will be created when **build** is executed. You can then put those files online using your favorite FTP client.

Whenever you want to upload a new version, run **npm run build** again even if the **/dist/** folder already exists.

We are not going to cover the setup of one of those "traditional" hosting solutions because we are going to use a more appropriate solution in the next section.

### Vercel

[Vercel](#) is one of those "modern" hosting solutions and features continuous integration (automatisation of testing, deployment and others development steps like this). It is very developer friendly and easy to setup.

You can use it for complex projects, but also for very simple "single page" websites like the ones we create within this course.

Before continuing, be aware that there is no partnership between me (Bruno Simon) and Vercel. I just like to use it for my small creative experiences.

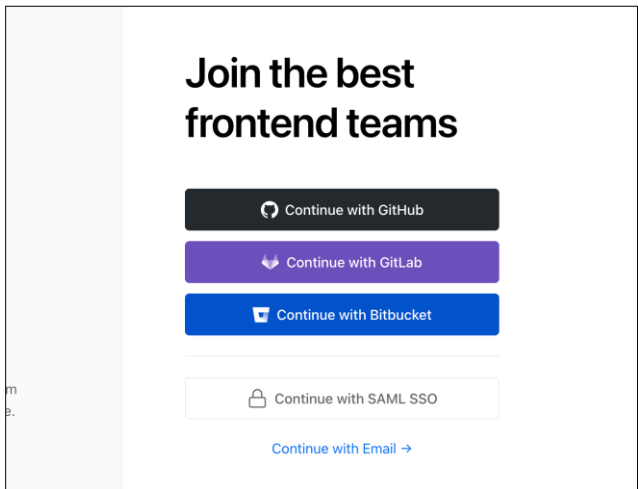
Also, there are other good alternatives that we should mention like [Netlify](#) and [GitHub Pages](#).

Be aware also that you might see slight differences between the rest of this lesson and your own experience with Vercel. It's quite a new solution and the developers keep improving the service.

Create an account

First, go to [vercel.com](https://vercel.com) and create an account.

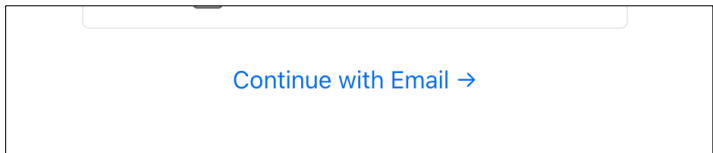
You can choose between different login methods such as **GitHub** connect, **GitLab** connect, **Bitbucket** connect or classic **Email** access.



The "connect" choices might be good options because one of the great features of Vercel is that it allows continuous integration as a versioning solution. If you don't know what [GitHub](#), [GitLab](#) and [Bitbucket](#) are, they are hosting solutions for your Git repositories. In other words, it's where most developers save their code.

If you are familiar with them, you can totally go for it. This will simplify the process and Vercel will automatically fetch your repository, help you setup the live version and automatically update it when you push new versions on the repository. You can even choose a specific branch.

For those of you who are not familiar with Git hosting solutions, don't worry, we are going to keep this lesson accessible to most students and choose the **Email** solution. Even if you are currently using Git, this solution will still work fine. Click the **Email** link.



Then enter your email address and follow the steps to create your account.

Add Vercel to your project

Vercel is available as an [NPM module](#) that you can install globally on your computer or as a dependency on your project. We are going to add it to the project so that, if we want to setup the project on another computer or we want to share it with another developer, we don't have to install anything on that computer.

In the terminal of your project, run **npm install vercel**. After a short while, the installation will be complete, but you will probably notice that there are some vulnerabilities. The terminal will instruct you to run an audit to fix them. You can usually ignore these warnings since they are probably false positives.

Although we added Vercel as a project dependency, instead of globally, it's still not available in the terminal. Your NPM scripts will have access to it, but you will first need to make the following change.

In **package.json**, in the **"scripts"** property, add a new script named **"deploy"** and write **"vercel --prod"** as the value (don't forget the **,** after the **"dev"** script):

```
{
  "scripts": {
    // ...
    "deploy": "vercel --prod"
  },
}
```

If you use the **"vercel"** command without **—prod**, the code will be published on a preview URL so that you can test it before going to production. Though it is an interesting feature, we don't need a preview version.

From now on, if you want to deploy your project online, you can simply run **npm run deploy** in the terminal.

### Deploy for the first time

The first time you want to deploy your project, Vercel will ask you for some information in order to connect to your account and setup the project.

In the terminal, run **npm run deploy**

If earlier, you chose to connect with an email, use the **up** and **down** arrows to select **Continue with Email** and press **Enter**:

```
> deploy
> vercel

Vercel CLI 23.1.2
> > No existing credentials found. Please log in:
> Log in to Vercel
  Continue with GitHub
  Continue with GitLab
  Continue with Bitbucket
> Continue with Email
  Continue with SAML Single Sign-On
  Abort
```

Then write your email:

```
Vercel CLI 23.1.2
> > No existing credentials found. Please log in:
> Log in to Vercel email
> Enter your email address: 
```

Vercel will send you an email like earlier. Click on the link available in that email and the terminal should automatically continue to the next step.

At this point, you should be prompt with a confirmation to setup and deploy the current folder. Choose **Y** and press **Enter**.

You can have multiple scopes and teams associated with your account. If you've just created your account, there should be only one. Choose it by pressing **Enter**

In our case, we are creating a new project and we don't want to associate our code with an existing project. Write **n** and press **Enter**:

```
? Link to existing project? [y/N] n
```

Vercel will try to guess the project name, but you can change it to anything you want. You can use lower case, numbers and dashes. Press **Enter**.

If you are executing that code at the root of the project folder, you can keep **./** and press **Enter**:

```
? In which directory is your code located? ./
```

Vercel has default settings for things like the build command or the bundle destination folder. We need to override those. Write **y** and press **Enter**:

```
No framework detected. Default Project Settings:
- Build Command: `npm run vercel-build` or `npm run build`
- Output Directory: `public` if it exists, or `.`
- Development Command: None
? Want to override the settings? [y/N] y
```

Vercel's default **Build Command** is **npm run build** and it's the same for us. But Vercel's default **Output Directory** is **public** while ours is **dist**.

Use the **up** and **down** arrows to move the cursor on **Output Directory** and press **Space** to check it. Then, press **Enter** to go to the next step:

```
? Which settings would you like to overwrite (select multiple)?
  □ Build Command
>  ■ Output Directory
  □ Development Command
```

Write **dist** and press **Enter**:

```
? What's your Output Directory? dist
```

That's it, your website should start deploying on Vercel.

Wait a minute or two and Vercel should prompt you with the URL to the live version.

Vercel should automatically copy that URL to your clipboard. You can paste it into your favorite browser and test the live version.